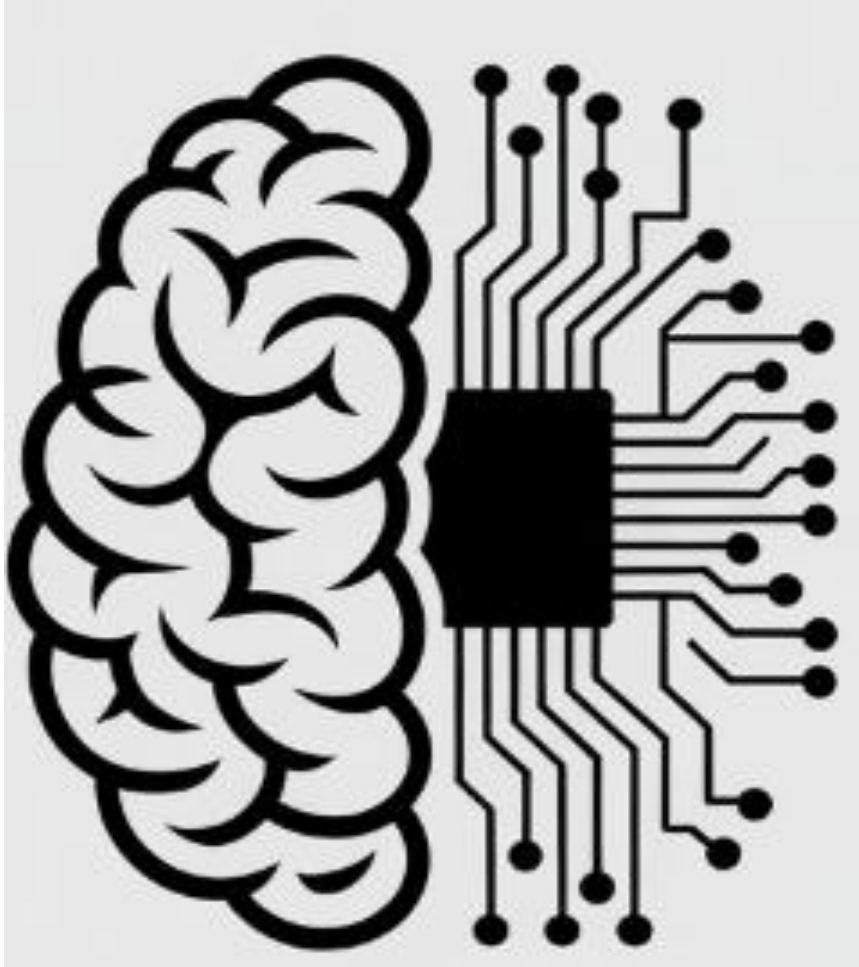




Blind Search

Dr. Sobia Tariq Javed

Problem Solving Agent

- Steps
 1. Goal formulation
 2. Problem formulation
 3. Search
 4. Execute

1. Goal Formulation

- **Goal formulation** is the process of defining what an agent aims to achieve based on its current state and the criteria used to measure success. It enables the agent to focus on a specific objective and determine the most appropriate sequence of actions required to achieve that goal.

1. Goal Formulation-Example

- An agent is currently at location A in a map. Its task is to reach location B with the shortest possible distance.
 - **Current state:** Agent is at location A
 - **Goal:** Reach location B
 - **Success measure:** Minimum distance or minimum time
 - **Result of goal formulation:** The agent decides to find the shortest path from A to B using an appropriate search strategy.

2. Problem Formulation

- **Problem formulation** is the process of deciding what actions and states to consider, given a goal.
- **Initial State:** The agent is at location A on the map.
- **State Space:** All possible locations (nodes) on the map that the agent can occupy.
- **Actions:** Move from the current location to any directly connected neighboring location.
- **Goal Test:** The agent reaches location B.

State Search Space (or State Space)

- The state search space is the set of all possible states that a problem can have, along with the paths (transitions) between those states caused by valid actions.
- It includes every situation the system can be in and all the ways it can move from one situation to another.

Components of a State Search Space

- **State:** A specific configuration of the problem
- **Initial state:** The starting configuration
- **Goal state:** The desired final configuration
- **Successor functions (actions):** Rules that move the system from one state to another
- **Path:** Links between states created by applying operators

Problem State Search Space

A thick, hand-drawn style orange line underlining the text.

Eight-Tile puzzle

State Search Space: 2D array

7	2	4
5		6
8	3	1

Start State

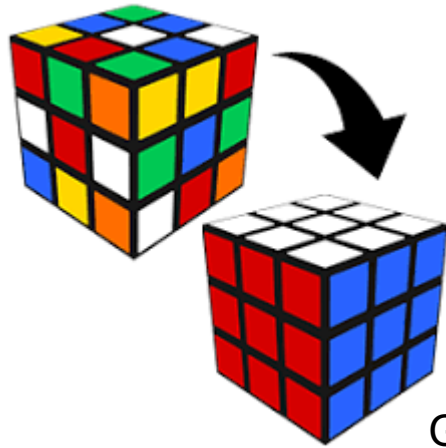
	1	2
3	4	5
6	7	8

Goal State

Action: Movement of blank space Left, Right, Up or Down

Rubrik cube

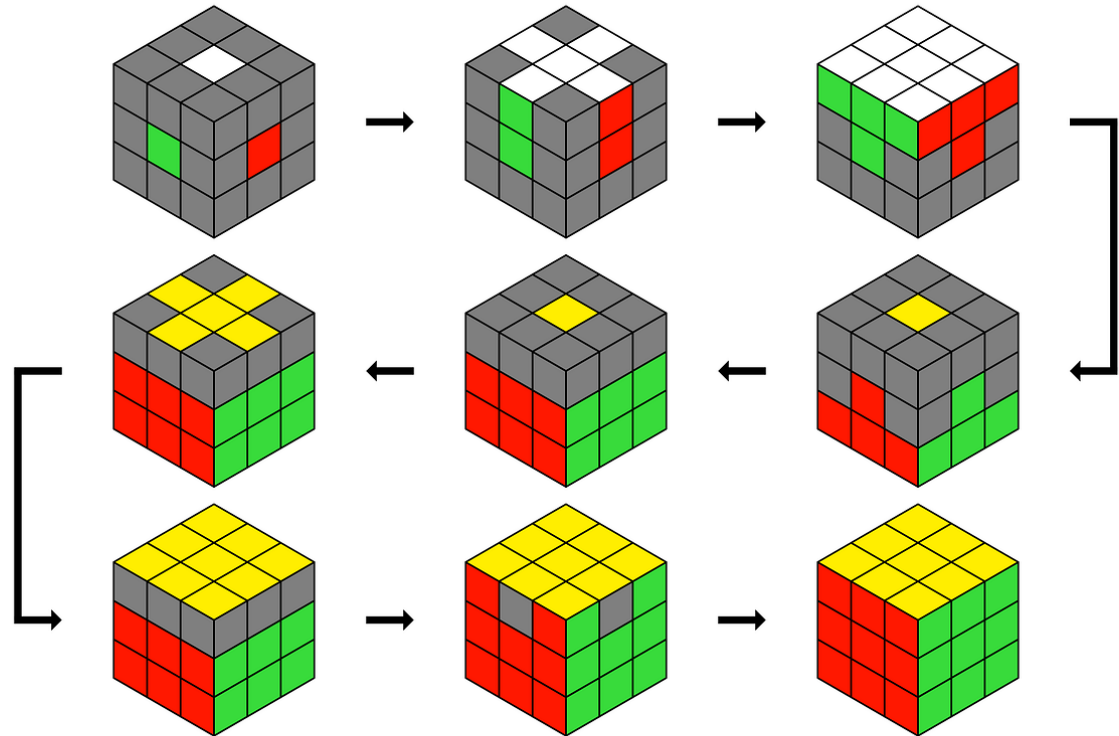
Initial State



Goal State

Actions

- U / U' / U2 – Rotate the upper face
- D / D' / D2 – Rotate the bottom face
- L / L' / L2 – Rotate the left face
- R / R' / R2 – Rotate the right face
- F / F' / F2 – Rotate the front face
- B / B' / B2 – Rotate the back face



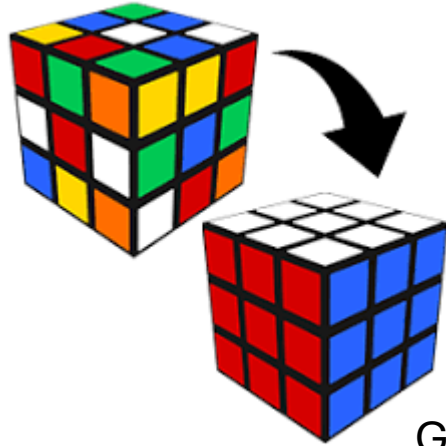
Rubrik cube

State Search Space: 3D array

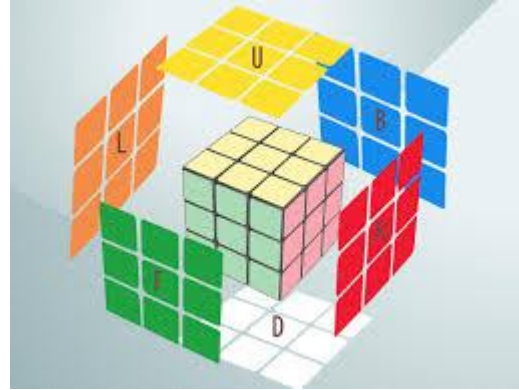
6 columns: each column containing one face of cube of size $n \times n$



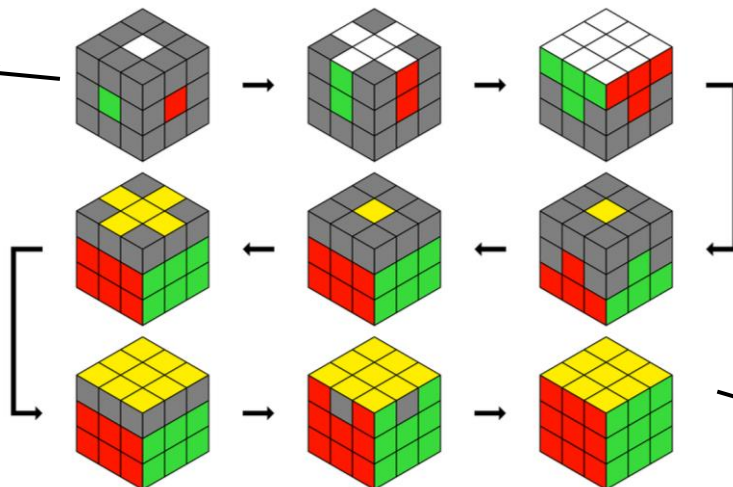
Initial State



Goal State

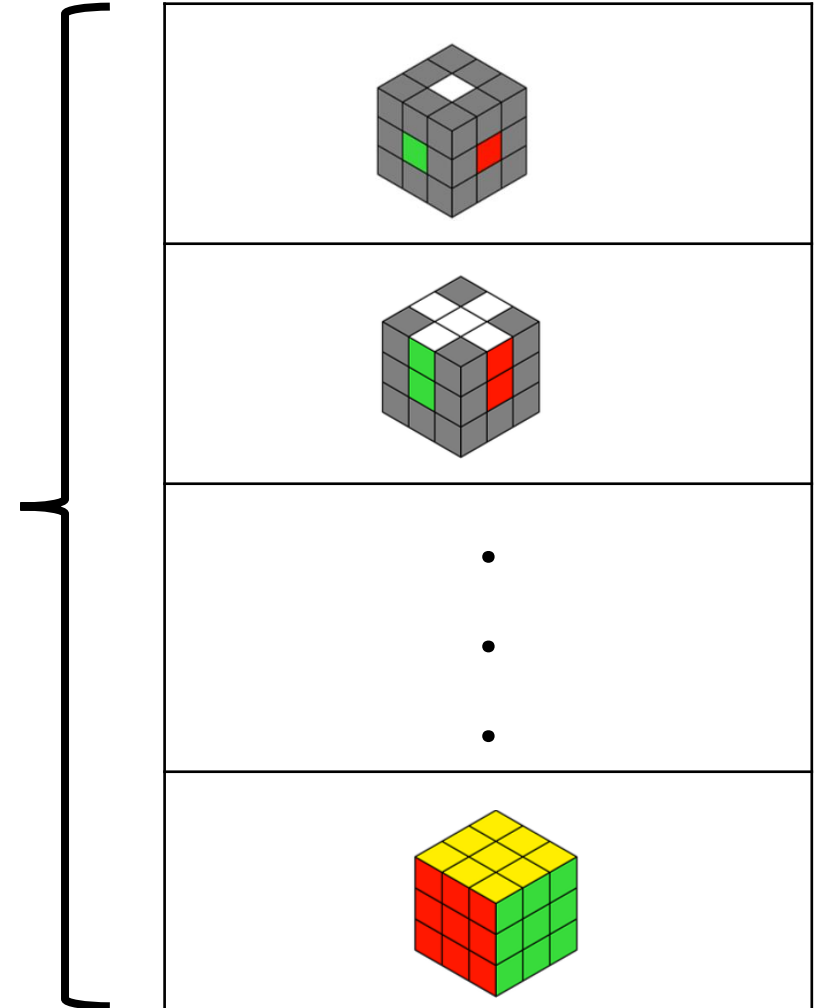


Initial State



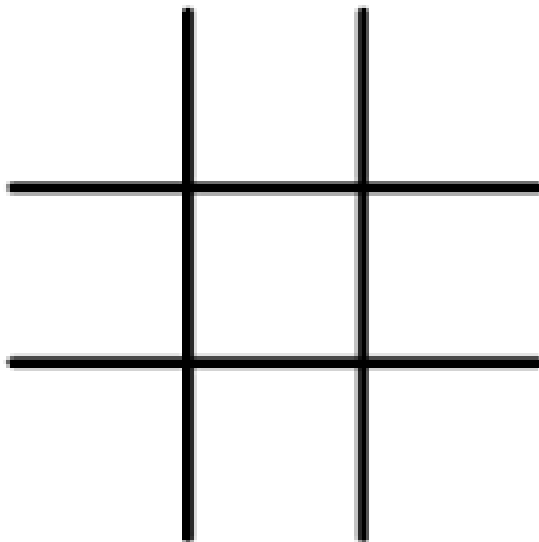
m rows: each row containing one whole cube (6 faces)

Goal State

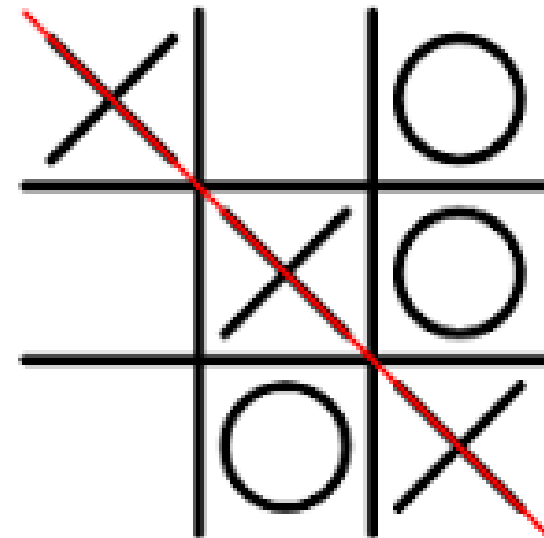


TiC-TAC-TOE

- A paper-and-pencil game for two players who take turns marking the spaces in a three-by-three grid, one with Xs and the other with Os. A player wins when they mark all three spaces of a row, column, or diagonal of the grid.



Initial



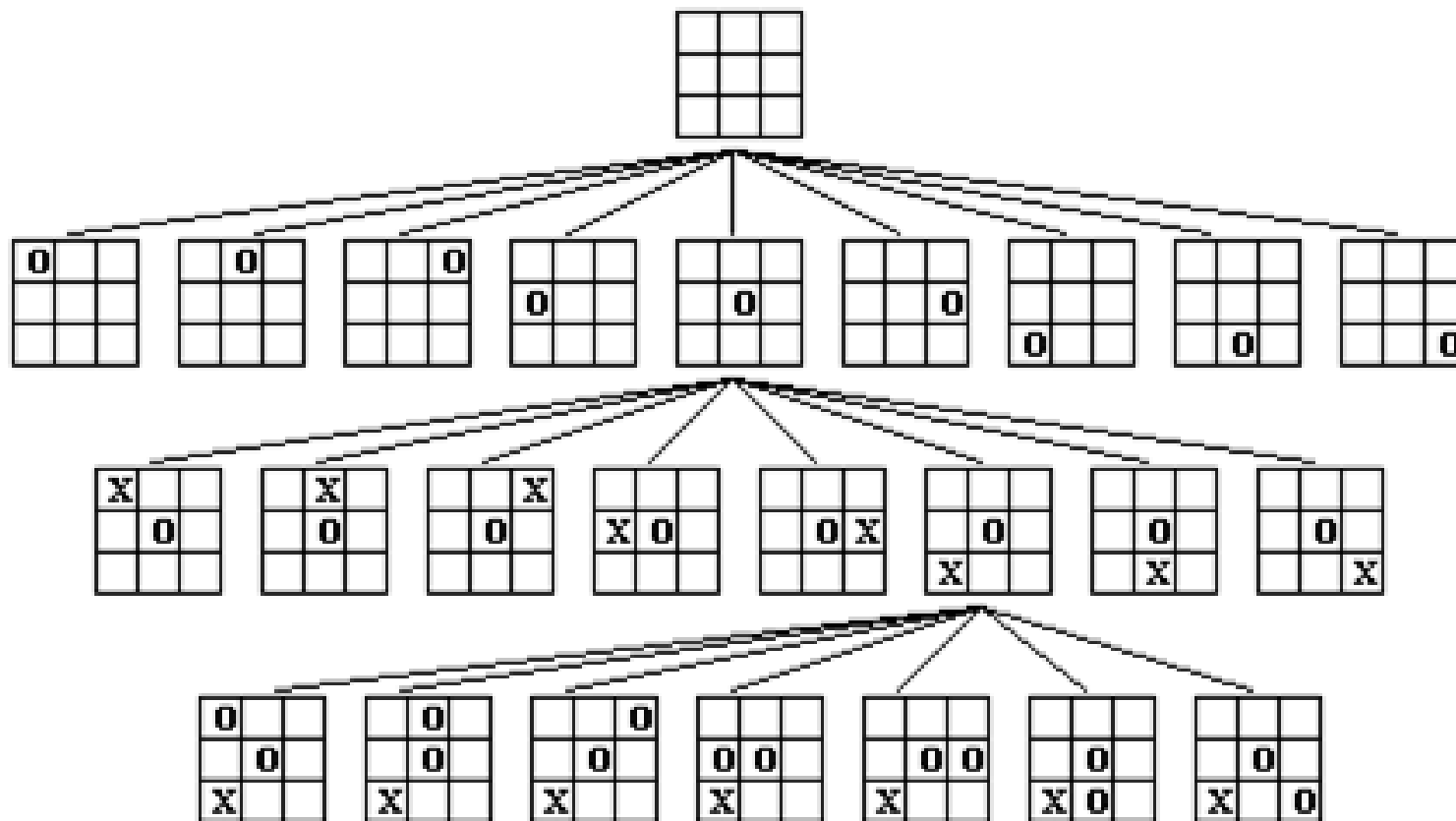
Goal

TiC-TAC-TOE

- Goal States
 - Player X wins (three Xs in a row, column, or diagonal), or
 - Player O wins, or
 - The game ends in a draw (board full, no winner).

Action: Fill one empty cell with the current player's symbol.

TiC-TAC-TOE



3. Search

- The process of *looking for a sequence of actions that reaches the goal is called search*.
- A search algorithm takes a problem as input and returns a solution in the form of an action sequence.

Strategies for State Space Search

A state space may be searched into two directions:

1. **Data Driven Search (forward chaining)**

From the given data of a problem instance towards a goal.

2. **Goal Driven Search (backward chaining)**

From a goal back to data.

Both searches, search the same state space graph, but the order and actual number of states searched can differ.

State Space Search Methods

1. Blind/Uninformed/Brute-force Search
2. Heuristic/Informed Search
3. Adversarial Search Algorithms (Game Playing)

State Space Search Methods

- **Blind/Uninformed/Brute-force Search**

1. Breadth First Search (BFS)
2. Depth First Search (DFS)
3. Iterative Deepening Search (IDS)
4. Uniform Cost Search (UCS)

State Space Search Methods

- **Heuristic/Informed Search**

1. Best First Search
2. A* Search

State Space Search Methods

- **Adversarial Search Algorithms (Game Playing)**

1. Minimax Search

- Alpha-Beta Pruning,
- Evaluation Functions

Measuring problem-solving performance

- **Completeness** → Will it find a solution?
- **Optimality** → Will it find the best solution?
- **Time Complexity** → How long will it take?
- **Space Complexity** → How much memory will it use?

4. Execution Phase

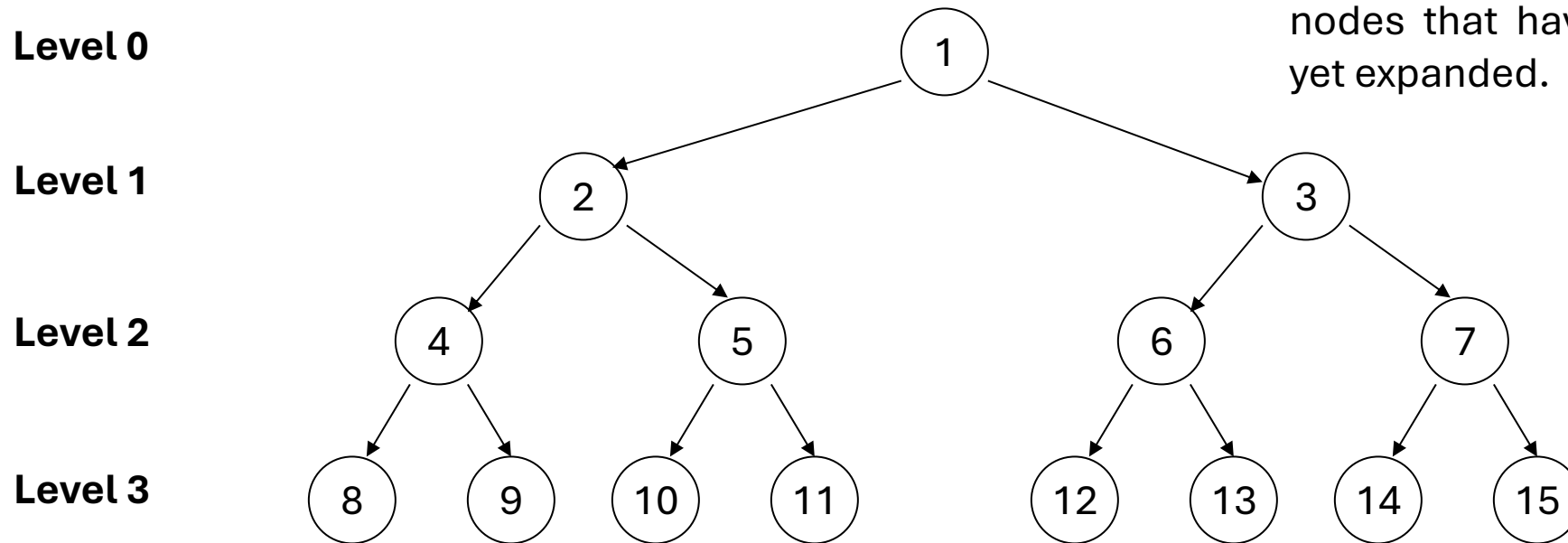
- Once a solution is found, the actions it recommends can be carried out. is called the **execution phase**

Blind/Uninformed/Brute-force Search

- Blind search methods do not use any information about the location of the goal in the search space.
- They explore the search space systematically and blindly, expanding nodes until a solution is found.
- The first solution found may not be optimal if multiple solutions exist.
- The search process is represented as a search tree, where nodes correspond to states.
- The algorithm begins at the initial state and explores the tree until it reaches a goal state.
- Different blind search methods are distinguished by how they traverse the search tree (e.g., BFS vs. DFS).

Blind/Uninformed/Brute-force Search

- Consider a tree



In **search algorithms**, the **frontier** (also called the **open list or fringe**) is the set of nodes that have been generated but not yet expanded.

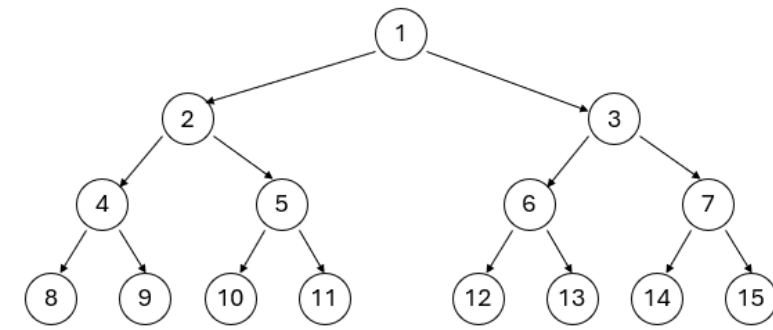
Closed list/ explored set is a set of all the states that have already been expanded.

d = Maximum Level

h = Height of the Tree = Maximum Level + 1

b = number of maximum children

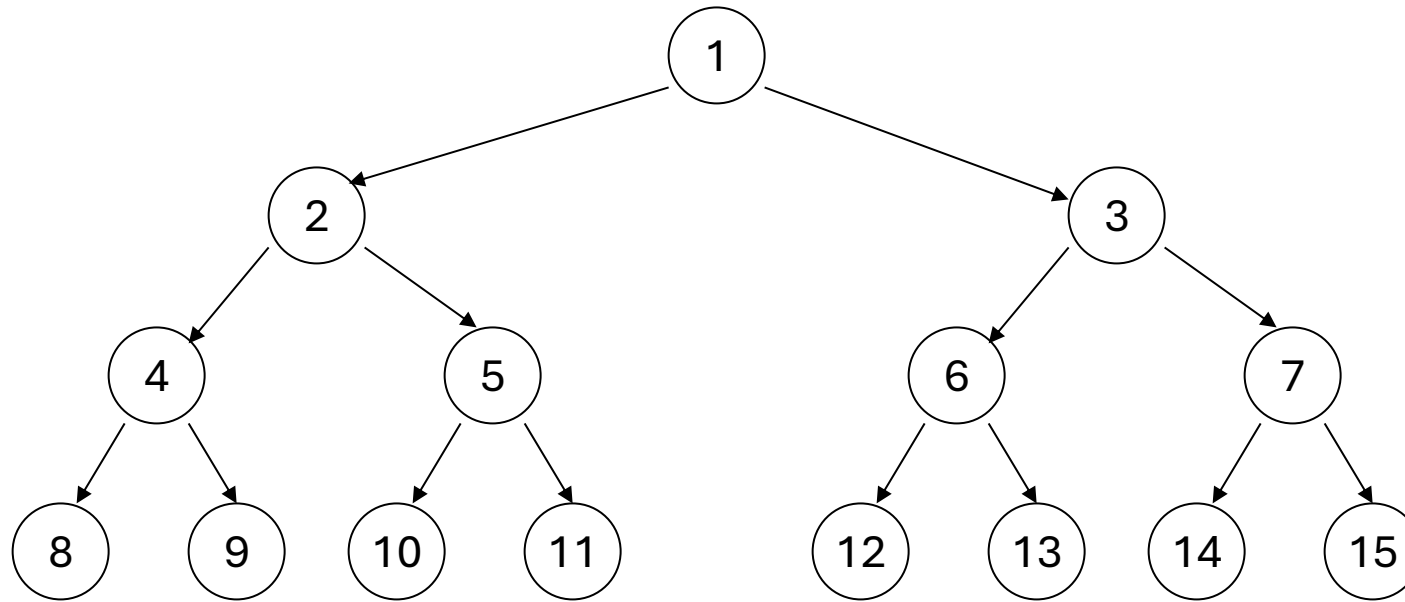
1. Breadth First Search (BFS)



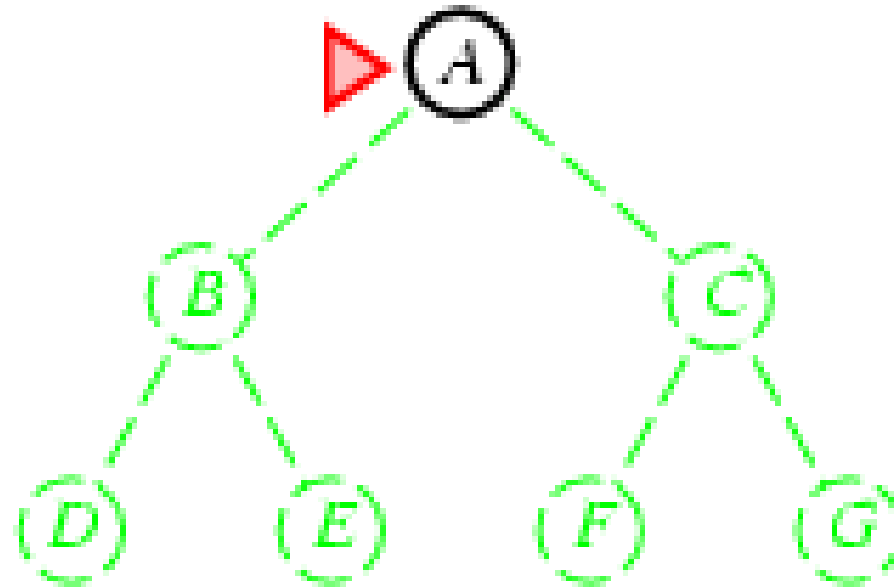
- It explores a search space level by level.
- Uses a queue (FIFO) data structure

Open & close List

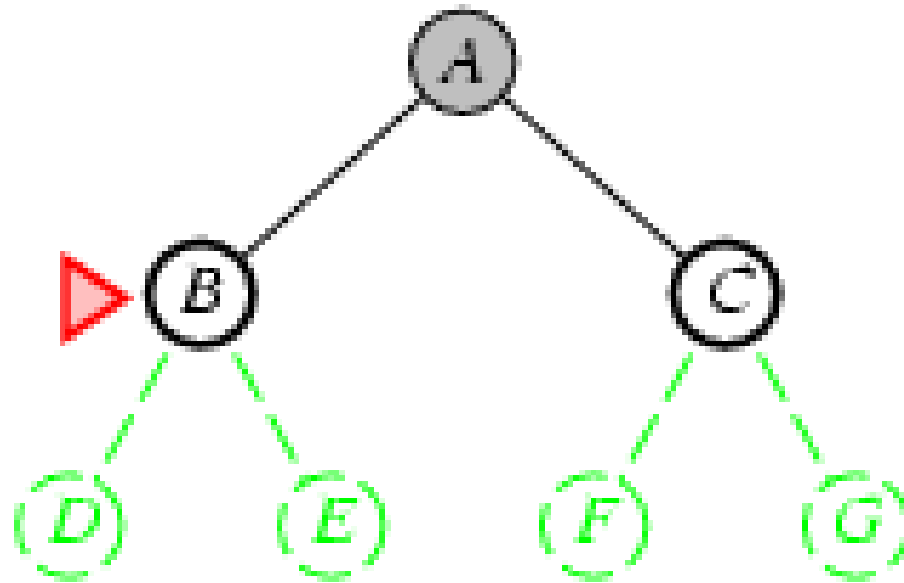
There is one slight tweak on the general graph-search algorithm, which is that the goal test is applied to each node when it is generated rather than when it is selected for expansion.



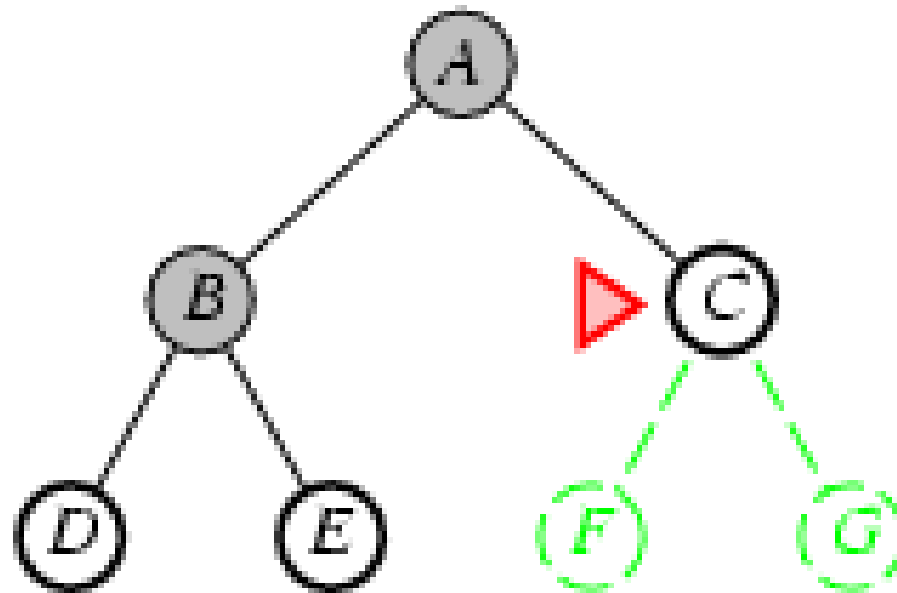
BFS



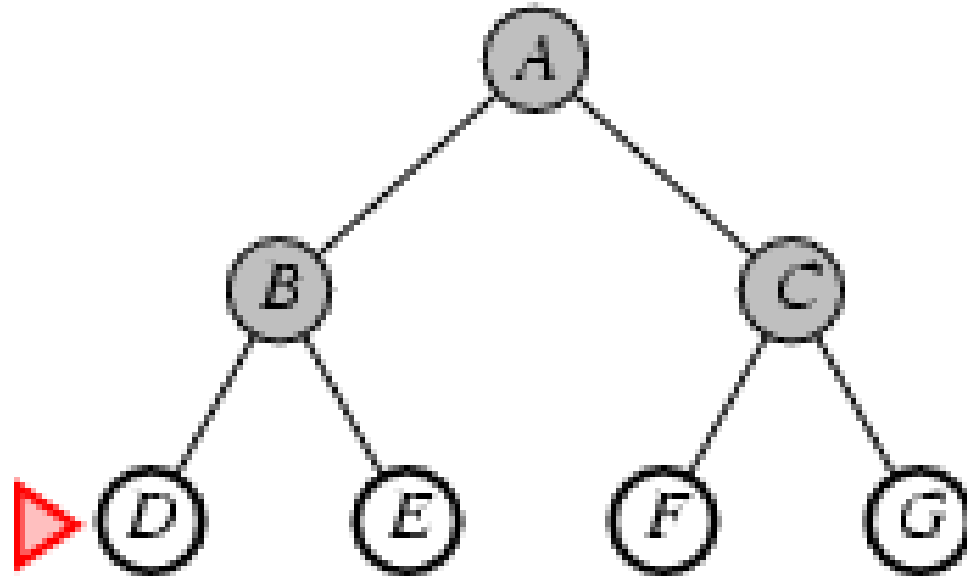
BFS



BFS



BFS



Vacuum Cleaner

State Search Space: 2D array

- Number of rooms = 2 (1 & 2)
- 0 = clean , 1 = dirt
- V = vacuum cleaner

Room 1	Room 2	Total dirty
D or C	D or C	Room
V presence		

- Initial State (I)

1	1	2
v		

or

1	1	2
	v	

- Goal State (G)

0	0	0
v		

or

0	0	0
	v	

Action: Move Left, Move Right, Clean (Suck dirt)

BFS-Vacuum Cleaner

Action: Move Left, Move Right, Clean (Suck dirt)

1	1	2
v		

I

0	0	0
v		

G

Eight-Tile puzzle

State Search Space: 2D array

2	8	3
1	6	4
7		5

Initial

1	2	3
8		4
7	6	5

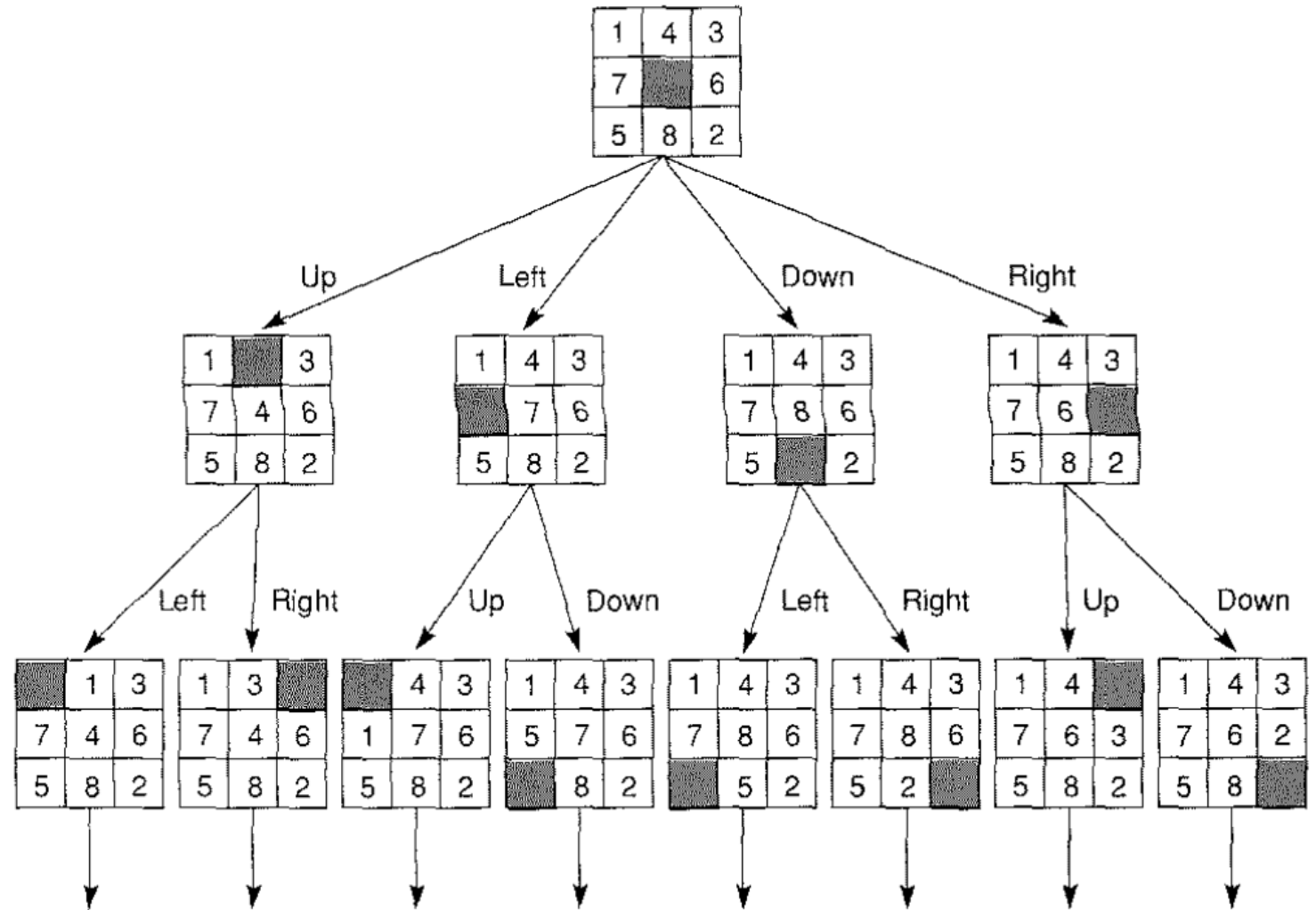
Goal 1

	1	2
3	4	5
6	7	8

Goal 2

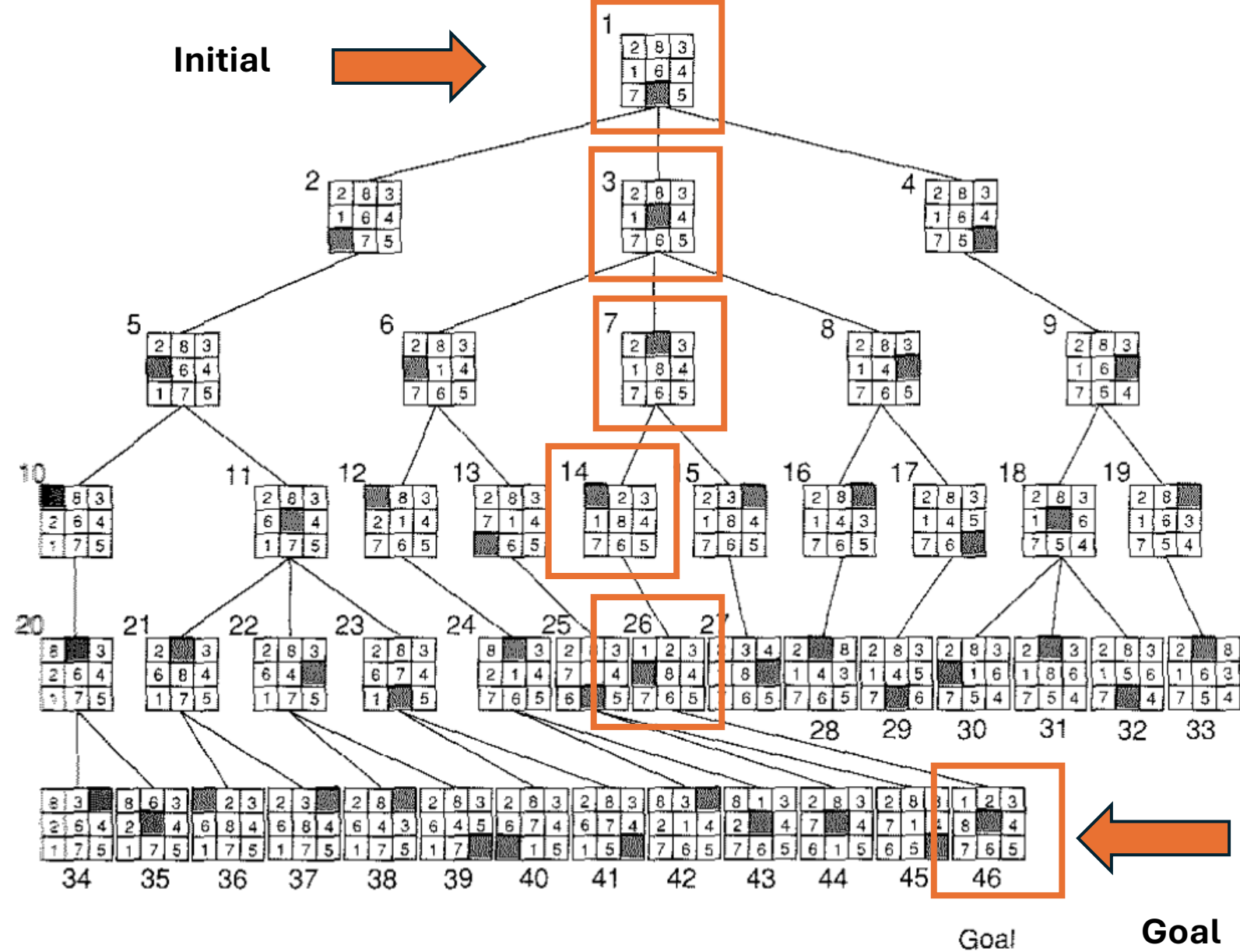
Action: Movement of blank space Left, Right, Up or Down

BFS- Eight-Tile puzzle



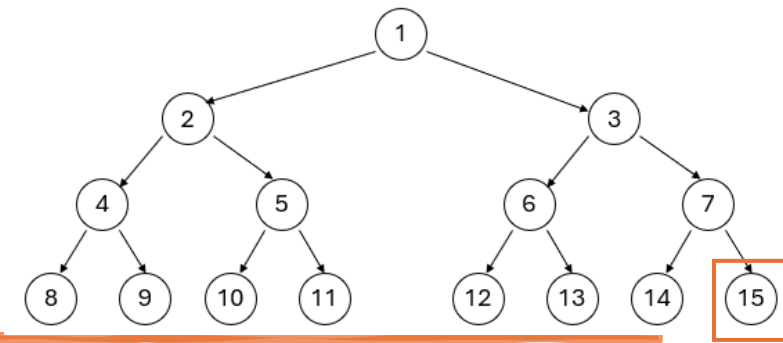
State space of the 8-puzzle generated by "move blank" operations.

BFS- Eight-Tile puzzle



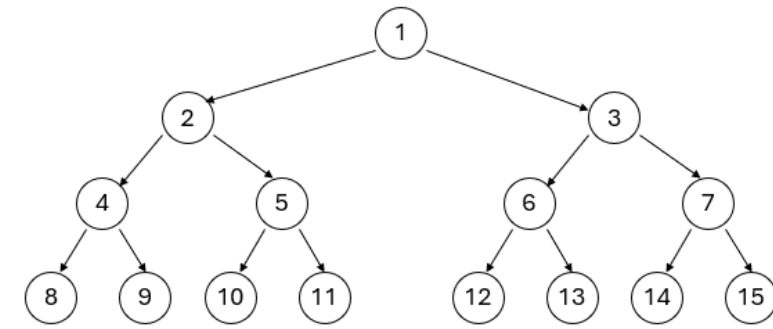
Breadth-first search of the 8-puzzle, showing order in which states were removed from open.

1. BFS – Time Complexity



- Assume a state space where every state has b successors
- Assume solution is at depth d
- Worst case: expand all but the last node at depth d
- Total number of nodes generated:
 $= b^1 + b^2 + \dots + b^d = O(b^d)$
- Time complexity = $O(b^d)$.

1. BFS – Space Complexity



- If $b=3$ and $d=4$
- $3^0 + 3^1 + 3^2 + 3^3 \leq 3^4$
- $1 + 3 + 9 + 27 \leq 81$
- $40 \leq 81$
- There will be $O(b^{d-1})$ nodes in the explored set and $O(b^d)$ nodes in the frontier.
- Every node generated must remain in memory so it will be same as time complexity
- Space complexity = **$O(b^d)$**

BFS

- **Complete?** Yes (if b is finite)

Time? $b^1 + b^2 + \dots + b^d = O(b^d)$

- **Space?** $O(b^d)$ (keeps every node in memory)
- **Optimal?** Yes (if cost = 1 per step)
- ***Space is the bigger problem*** (more than time)

Using Breadth-first Search

- **When is BFS appropriate?**

- space is not a problem
- it's necessary to find the solution with the fewest arcs
- although all solutions may not be shallow, at least some are

- **When is BFS inappropriate?**

- space is limited
- all solutions tend to be located deep in the tree
- the branching factor is very large

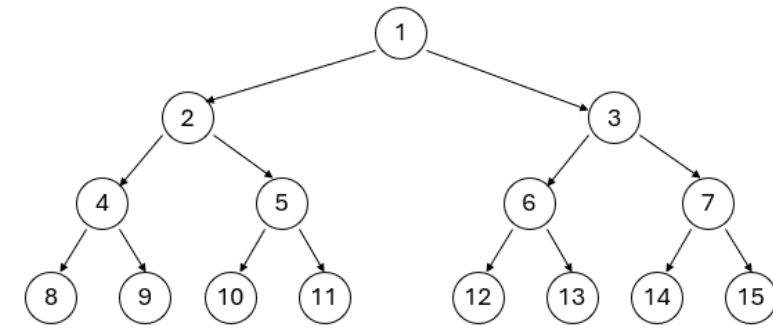
1. BFS

- Exponential-complexity search problems cannot be solved by uninformed methods.

Depth	Nodes	Time	Memory
2	110	.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes

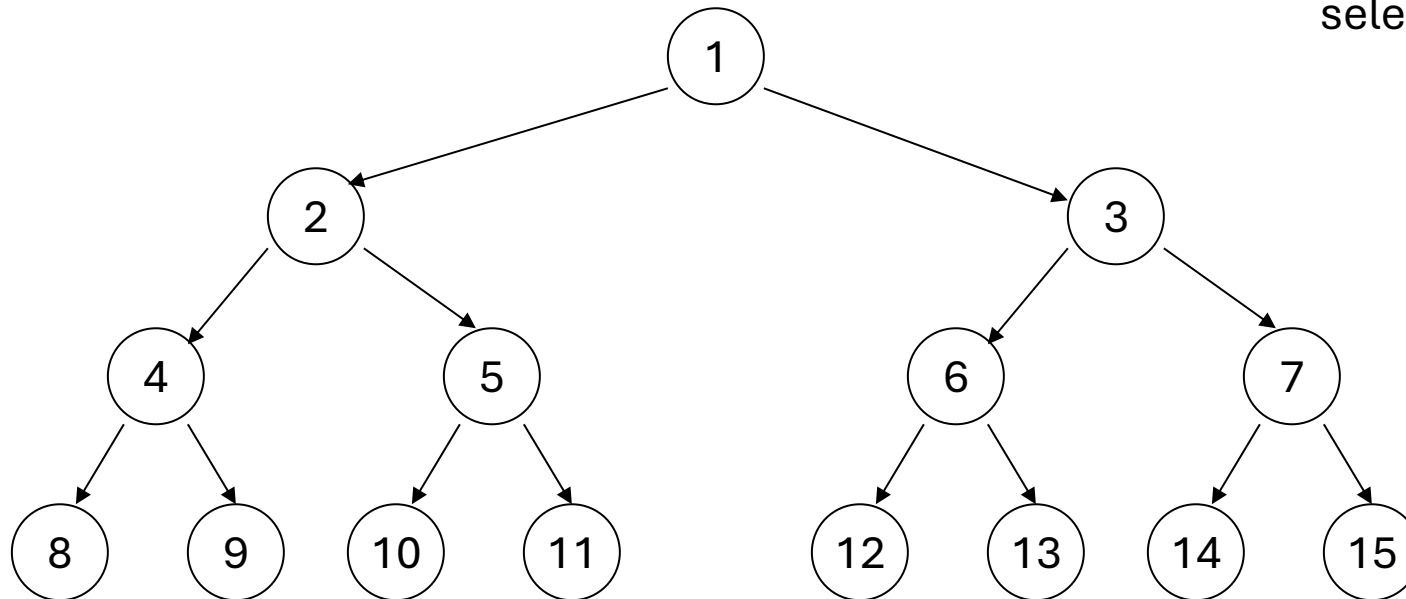
Figure 3.13 Time and memory requirements for breadth-first search. The numbers shown assume branching factor $b = 10$; 1 million nodes/second; 1000 bytes/node.

1. Depth First Search (DFS)



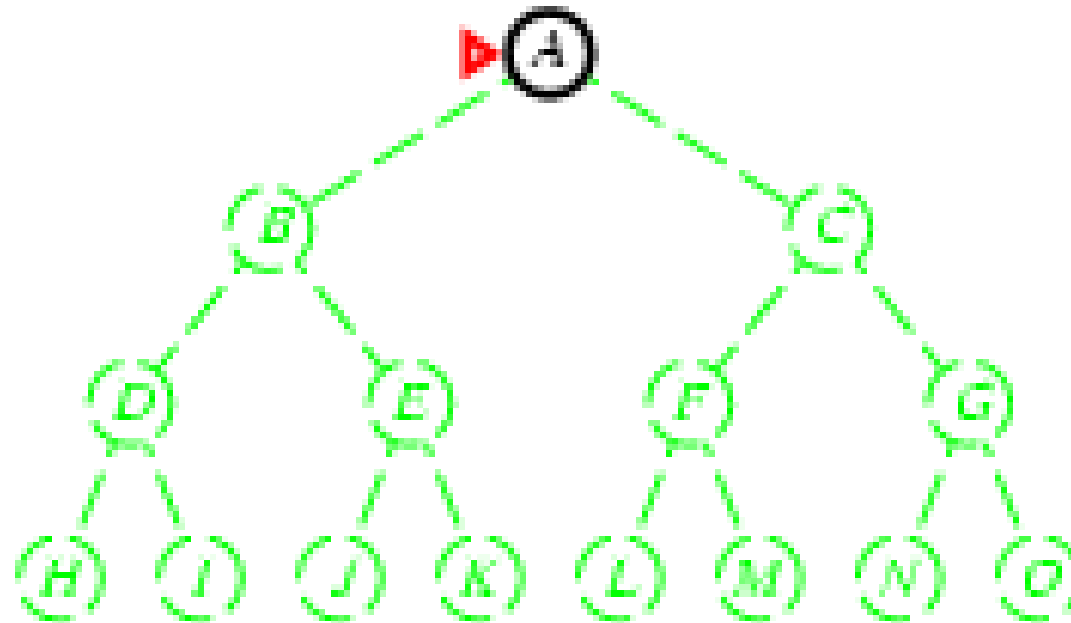
- It explores a search space by going as deep as possible along one path before backtracking.
- Uses a stack (LIFO) data structure

The goal test is applied to each node when it is selected for expansion.



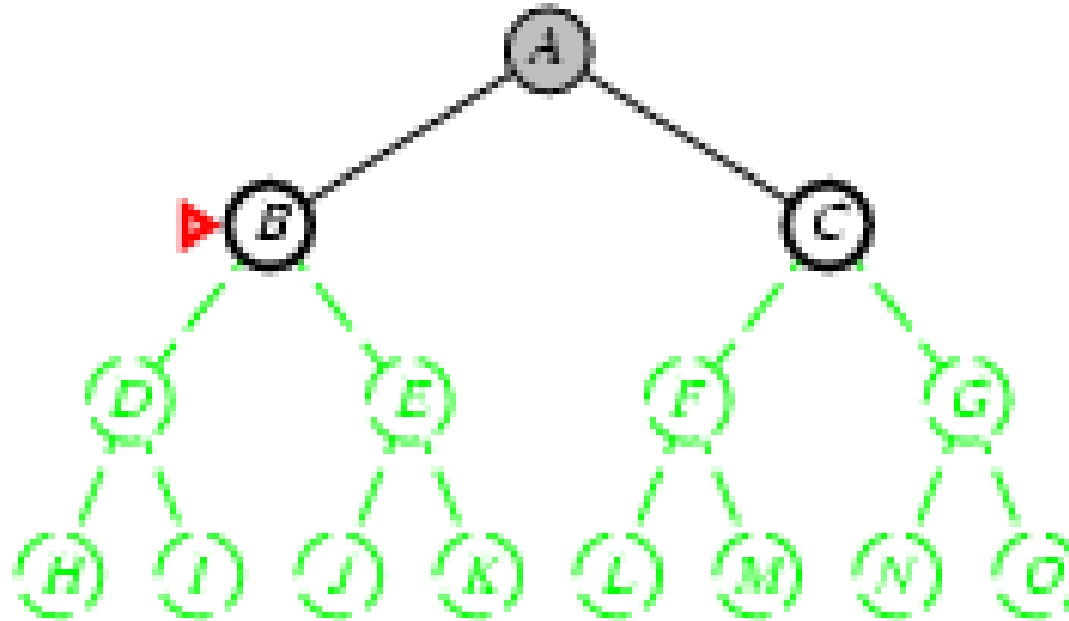
DFS

- The unexplored region is shown in light gray. Explored nodes with no descendants in the frontier are removed from memory.
- Goal is M



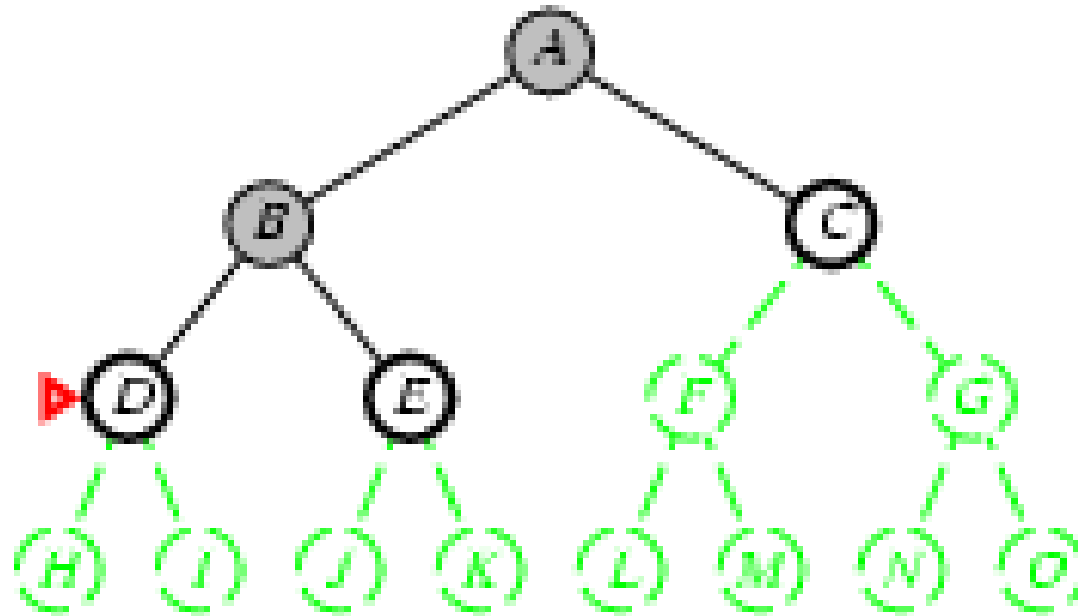
DFS

- The unexplored region is shown in light gray. Explored nodes with no descendants in the frontier are removed from memory.



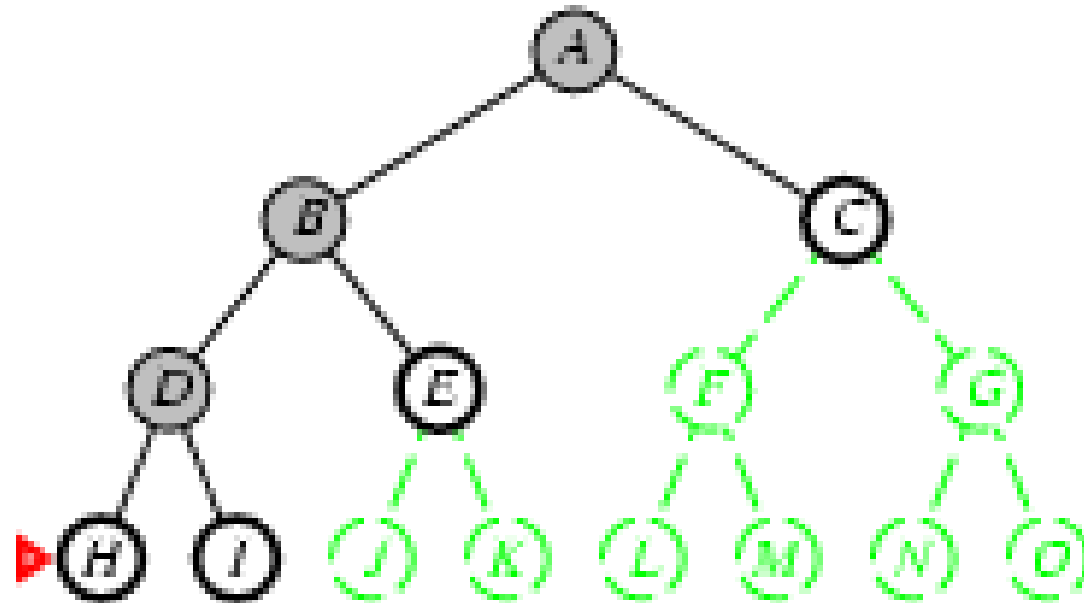
DFS

- The unexplored region is shown in light gray. Explored nodes with no descendants in the frontier are removed from memory.



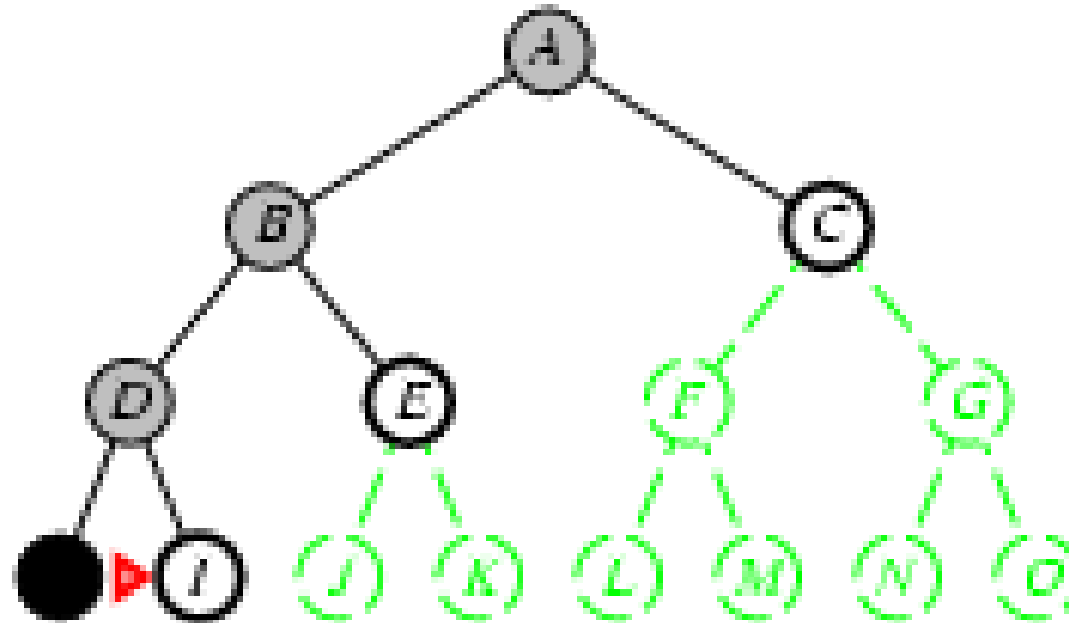
DFS

- The unexplored region is shown in light gray. Explored nodes with no descendants in the frontier are removed from memory.



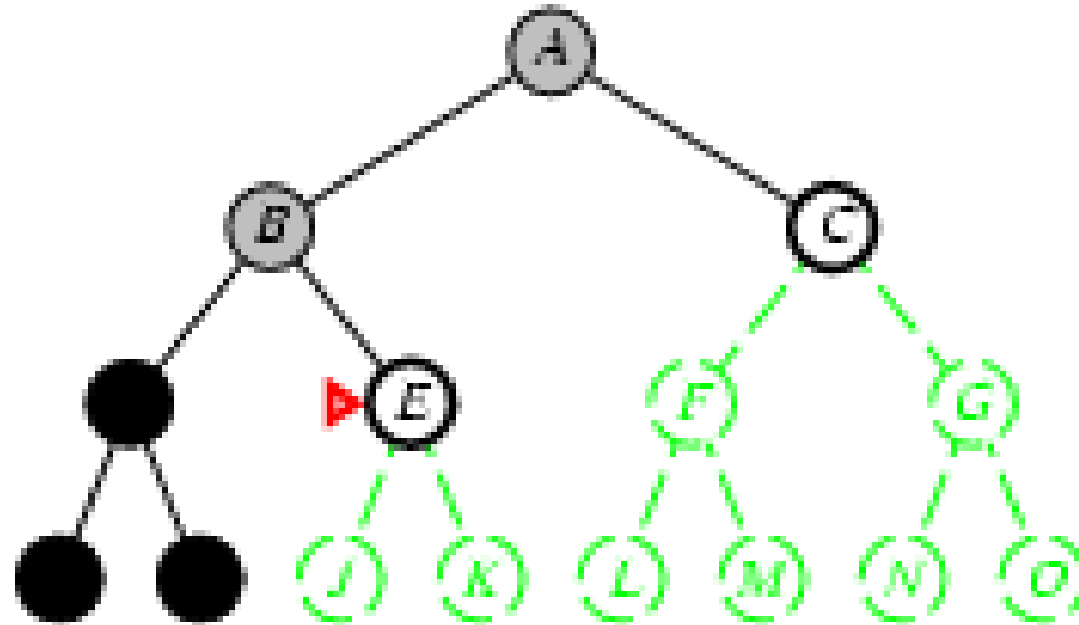
DFS

- The unexplored region is shown in light gray. Explored nodes with no descendants in the frontier are removed from memory.



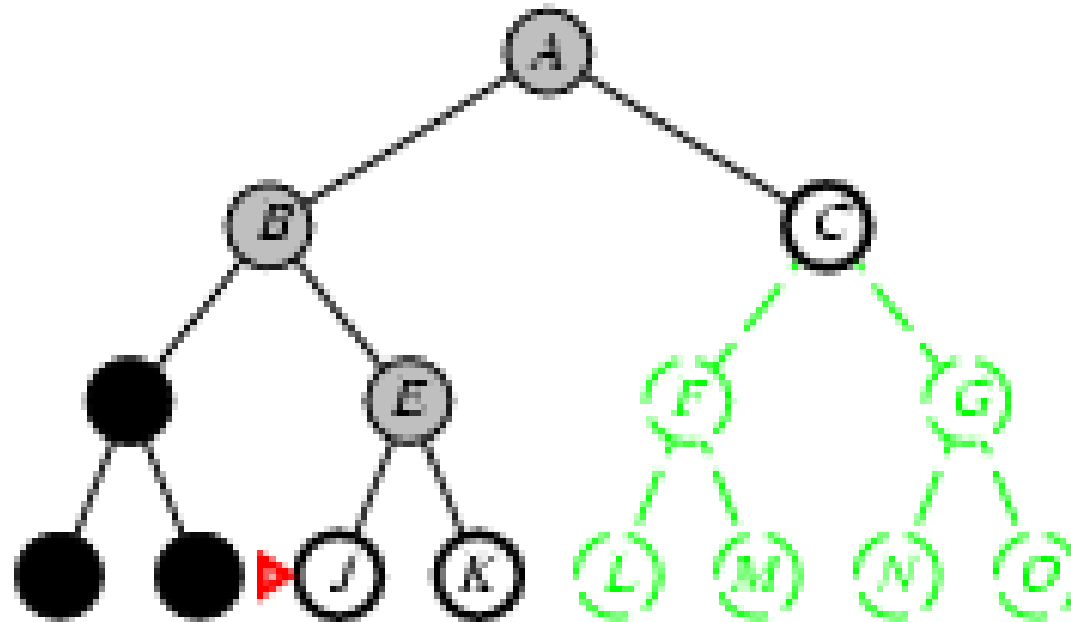
DFS

- The unexplored region is shown in light gray. Explored nodes with no descendants in the frontier are removed from memory.



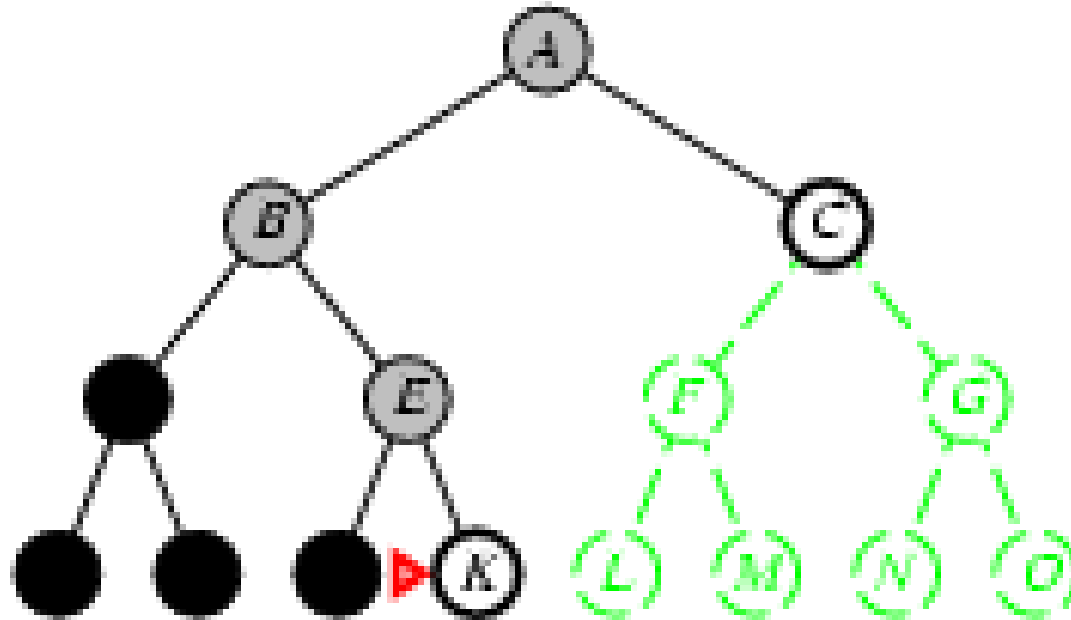
DFS

- The unexplored region is shown in light gray. Explored nodes with no descendants in the frontier are removed from memory.



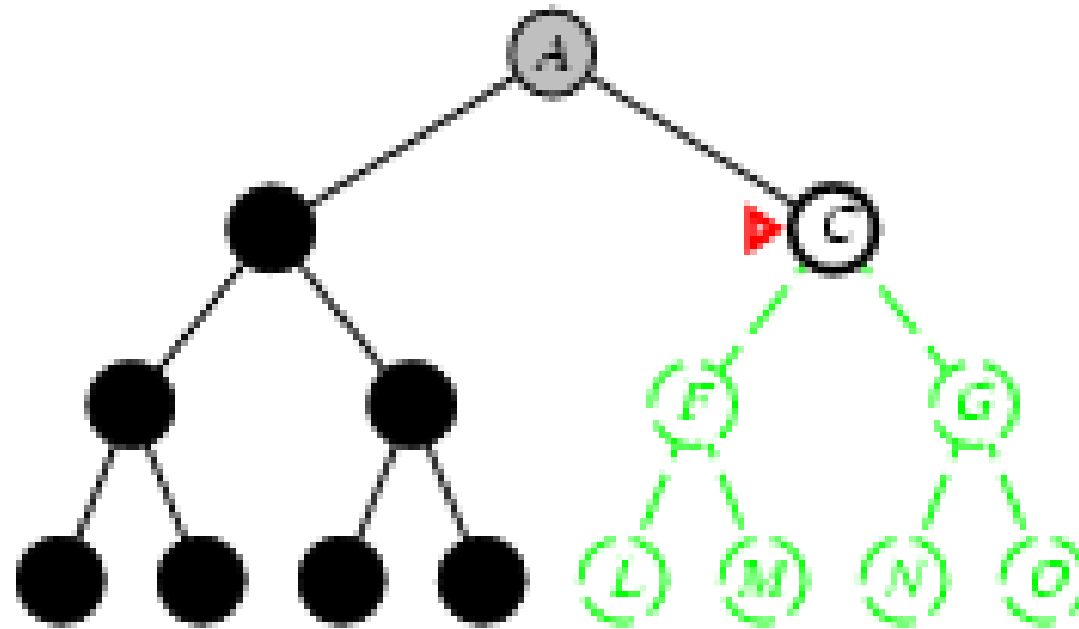
DFS

- The unexplored region is shown in light gray. Explored nodes with no descendants in the frontier are removed from memory.



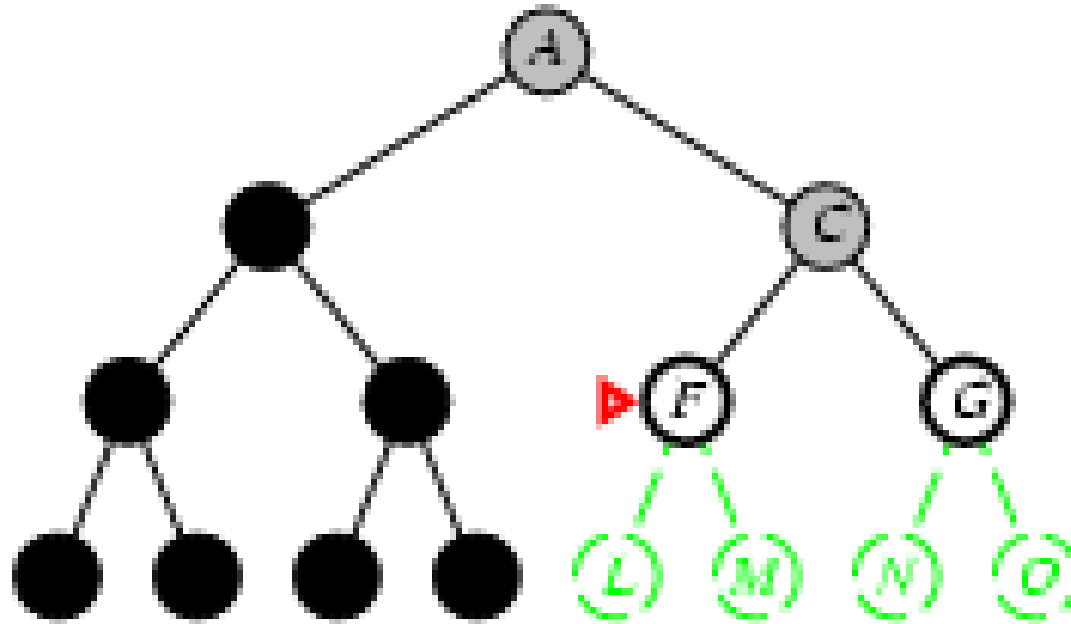
DFS

- The unexplored region is shown in light gray. Explored nodes with no descendants in the frontier are removed from memory.



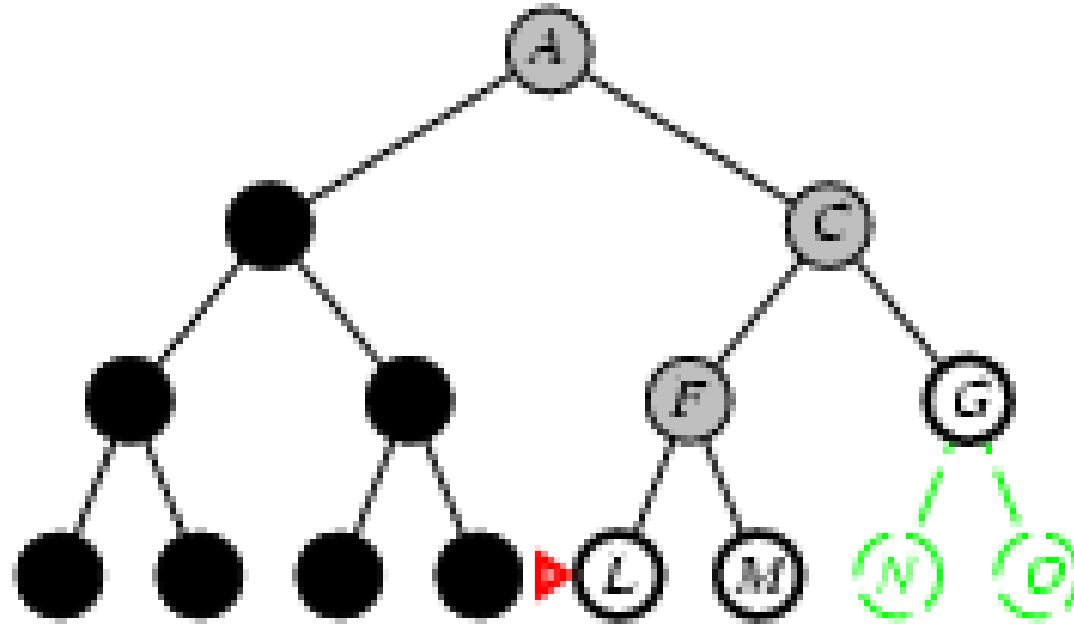
DFS

- The unexplored region is shown in light gray. Explored nodes with no descendants in the frontier are removed from memory.



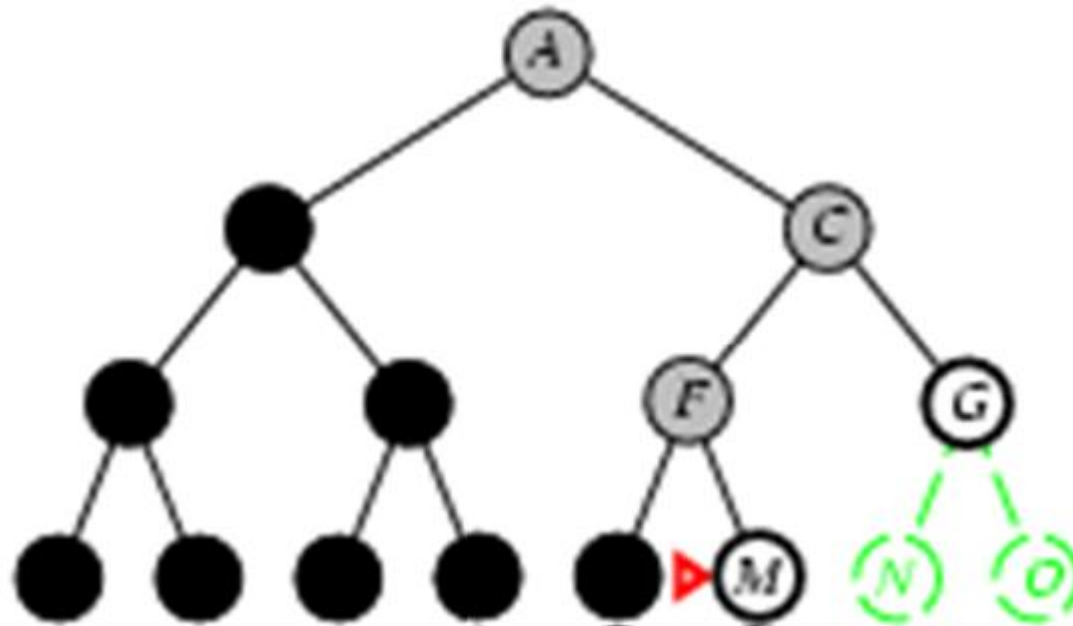
DFS

- The unexplored region is shown in light gray. Explored nodes with no descendants in the frontier are removed from memory.

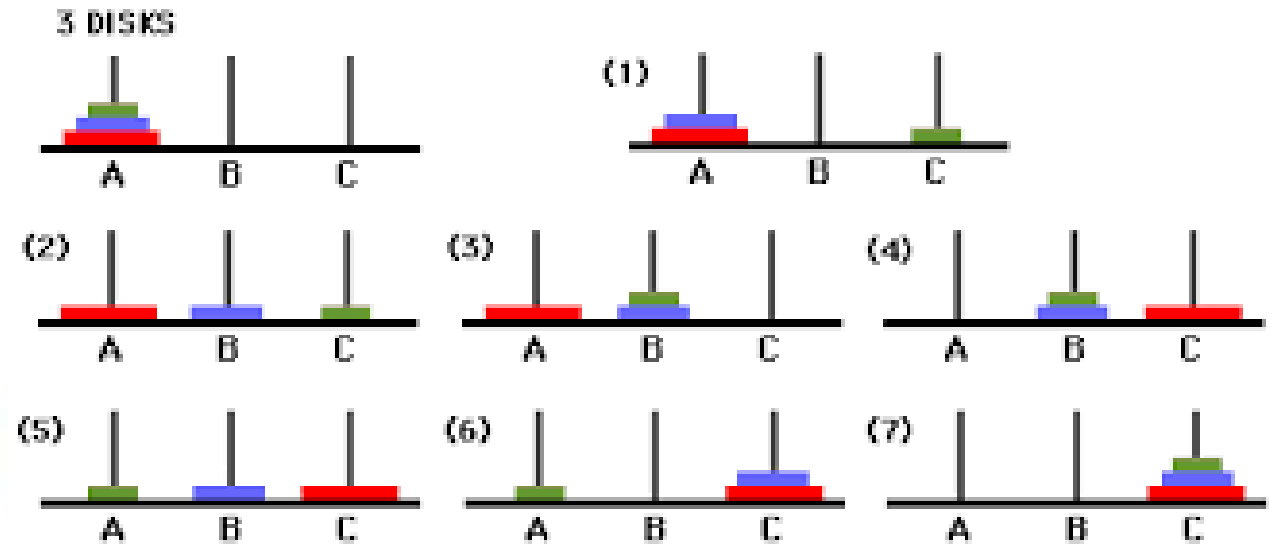
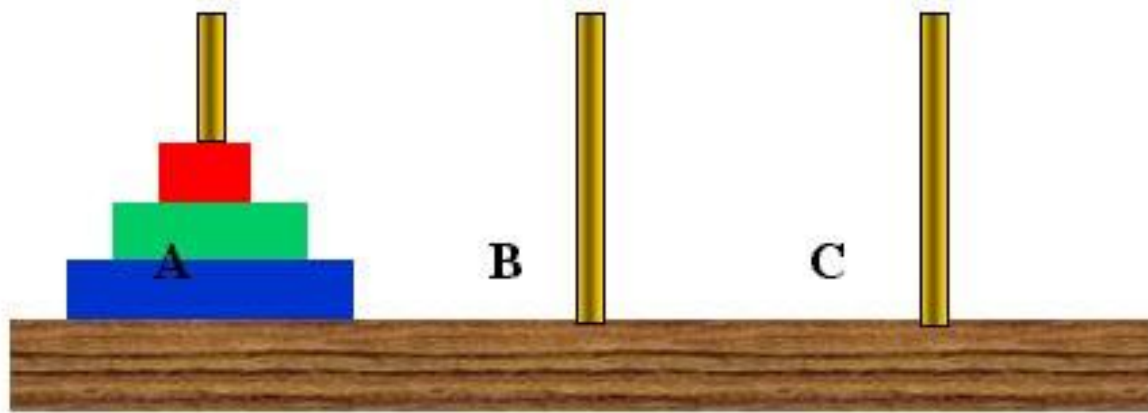


DFS

- The unexplored region is shown in light gray. Explored nodes with no descendants in the frontier are removed from memory.

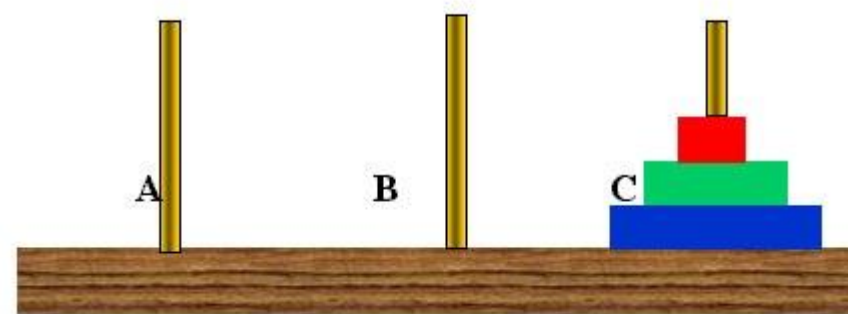
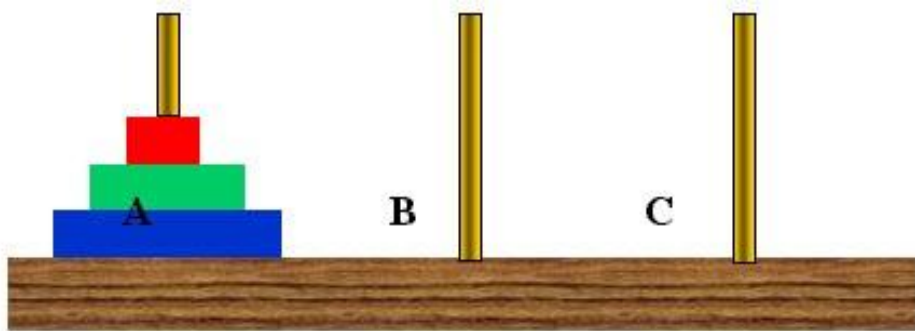


Tower of Hanoi



Tower of Hanoi

Action: move one disk at a time, never place a larger disk on a smaller one, and use a third auxiliary rod.



A	B	C
---	---	---

n	0	0
---	---	---

Initial

0	0	n
---	---	---

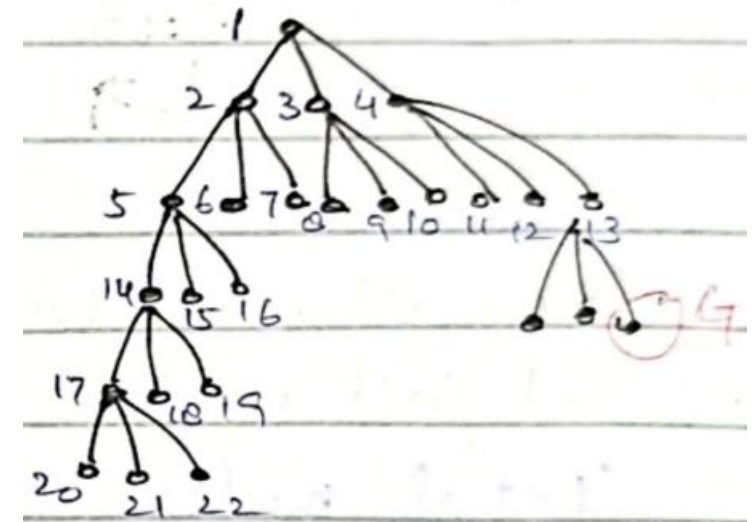
Goal

DFS

- On board

DFS

- Time Complexity: $O(b^m)$
- If m (maximal depth) is much larger than d (depth of shallowest solution) – time is terrible
- If there exist multiple solutions, DFS is faster than BFS
- Space Complexity: $O(bm)$



DFS

- **Complete?** No: fails in infinite-depth spaces,
- **Time?** $O(b^m)$: terrible if m is much larger than d
- m : maximum depth of any node, d =depth of the shallowest node
- **Space?** $O(bm)$
- **Optimal?** No

Real-World Scenario Where DFS Is Not Complete

- Imagine a web crawler that uses **Depth-First Search** to find a specific webpage.
 - The crawler starts from a homepage and keeps following the **first available link** on each page.
 - Some websites contain **endless links** (e.g., calendar pages, auto-generated URLs, or loops between pages).
 - The crawler continues going deeper into these links and **never returns** to explore other links.
 - The target webpage exists on a different branch, but it is **never reached**.

Depth-first Search: When it is appropriate?

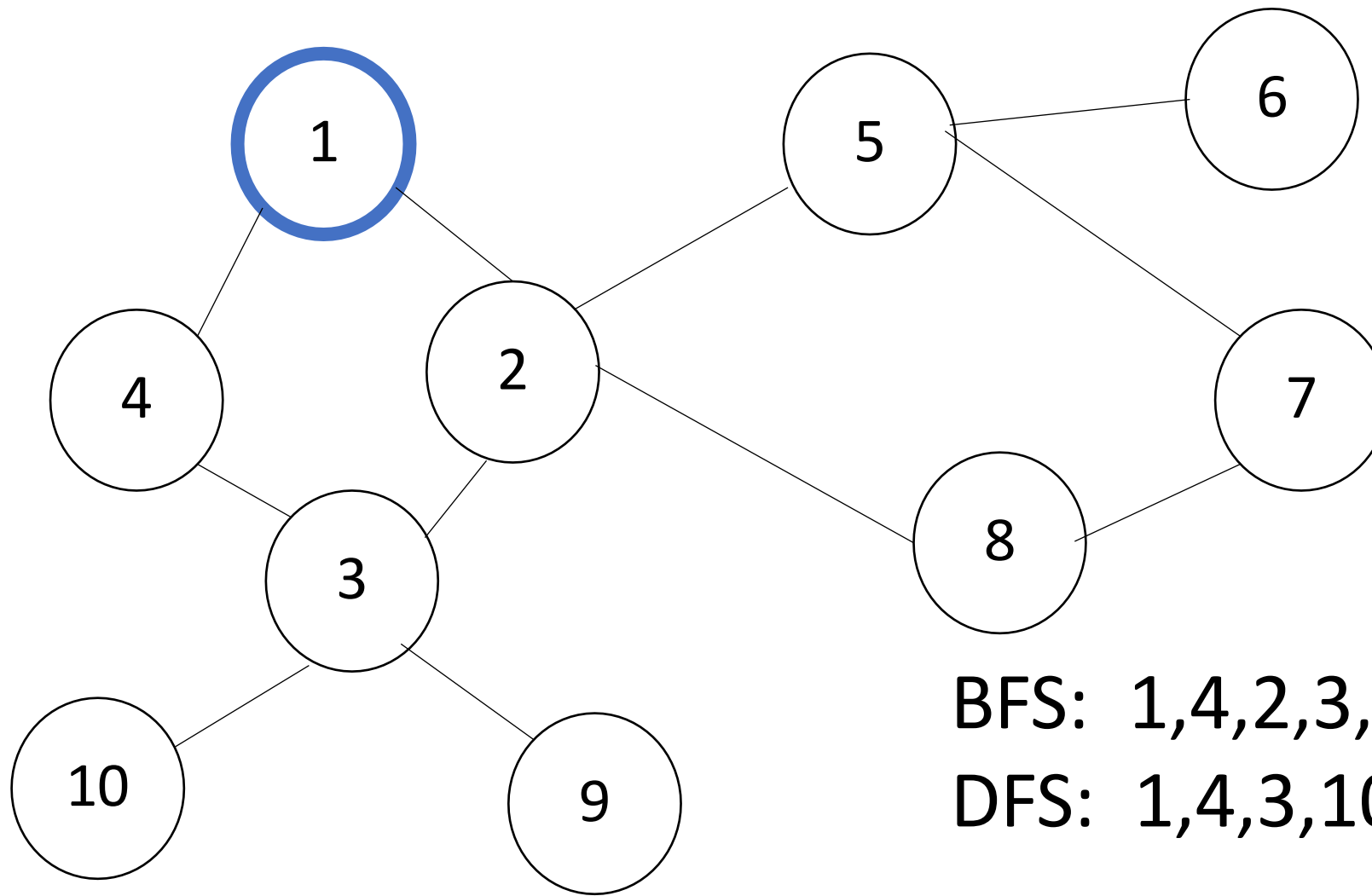
- **Appropriate**

- Space is restricted (complex state representation e.g., robotics)
- There are many solutions, perhaps with long path lengths, particularly for the case in which all paths lead to a solution

- **Inappropriate**

- Cycles
- There are shallow solutions

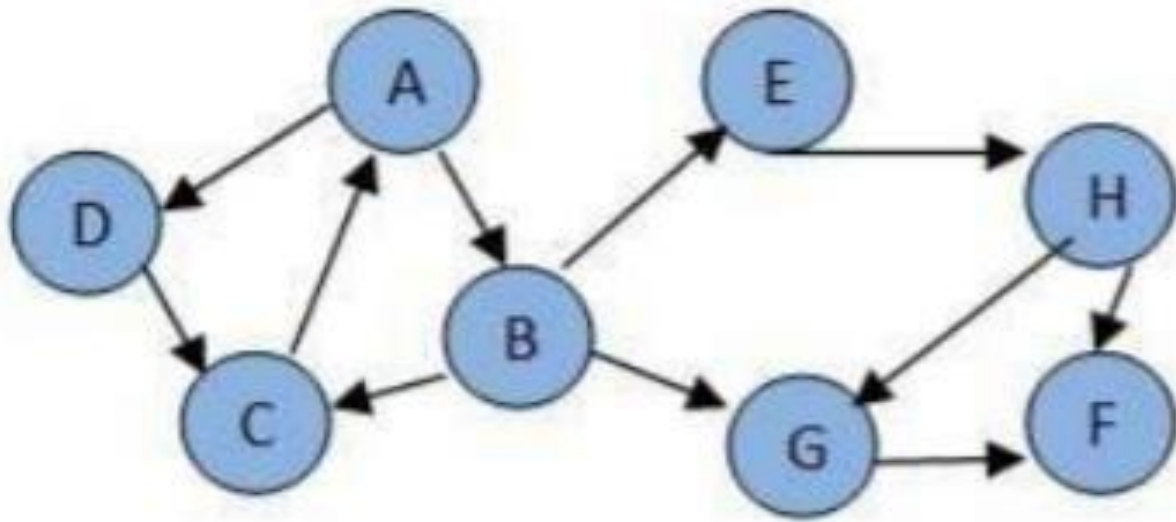
Apply BFS and DFS on following Graph



BFS: 1,4,2,3,5,8,10,9,7,6

DFS: 1,4,3,10,9,2,8,7,5,6

Apply BFS and DFS on following Graph



BFS: A,B,D,C,E,G,H,F

DFS: A,B,C,E,H,F,G,D

Homework #1

QUESTION: Given below the configurations of simplified Pacman game without ghosts. The objective of the game is to eat all of the dots placed in the maze. The Pacman can move in four directions (up, down, left, and right). Whenever Pacman enters in a cell with the dot it immediately consumes it. The Grey cells in the maze are blocked.

●		o	o
	■		
	■	o	■
	o	o	■

Initial State

	■		
	■		■
			■

Goal State

Part a.

Execute the state space search algorithms on the following maze where, numbers represent food cells and alphabets represents empty cells. You can build the tree with current position of Pacman instead of drawing complete states. You should create the nodes from left to right in **alphabetical order**.

●	D	1	2
A	■	E	F
B	■	3	■
C	5	4	■

Initial State

	D		
A	■	E	F
B	■		■
C			■

Goal State

1. Depth First Search (DFS)
2. Breadth First Search (BFS)

(Marks: 5)
(Marks: 5)

Iterative Deepening Search (IDS)

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution, or fail-  
ure
```

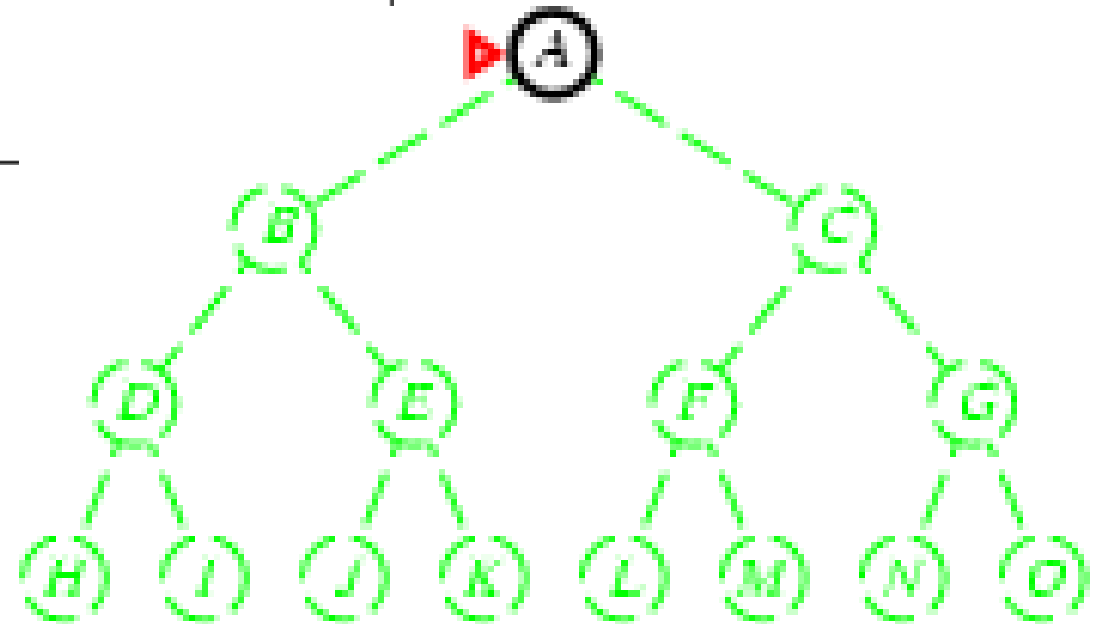
```
  inputs: problem, a problem
```

```
  for depth  $\leftarrow$  0 to  $\infty$  do
```

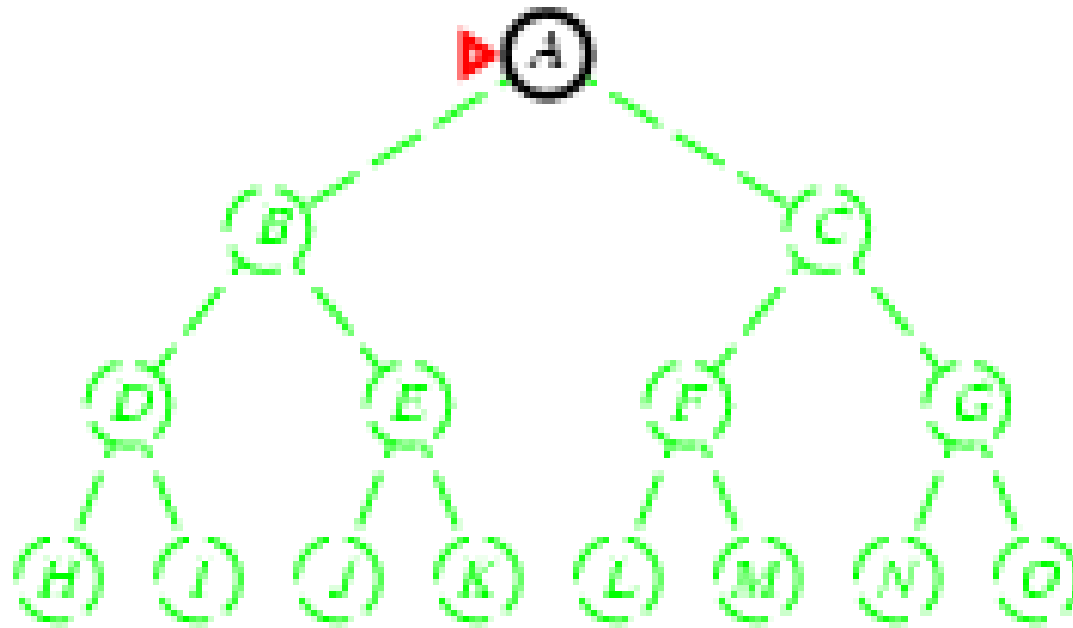
```
    result  $\leftarrow$  DEPTH-LIMITED-SEARCH(problem, depth)
```

```
    if result  $\neq$  cutoff then return result
```

Depth-Limited Search is a variant of Depth-First Search that explores nodes only up to a predefined depth limit.



Iterative Deepening Search (IDS)



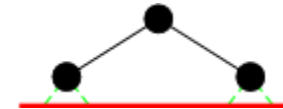
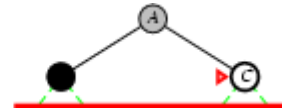
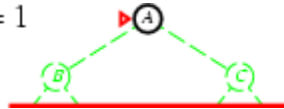
Iterative Deepening Search (IDS)

Limit = 0



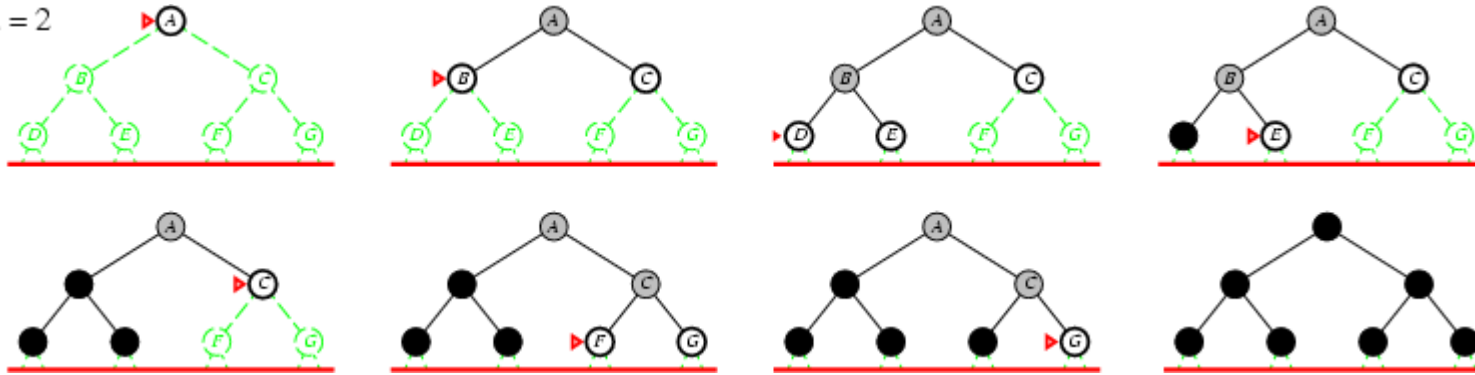
Iterative Deepening Search (IDS)

Limit = 1



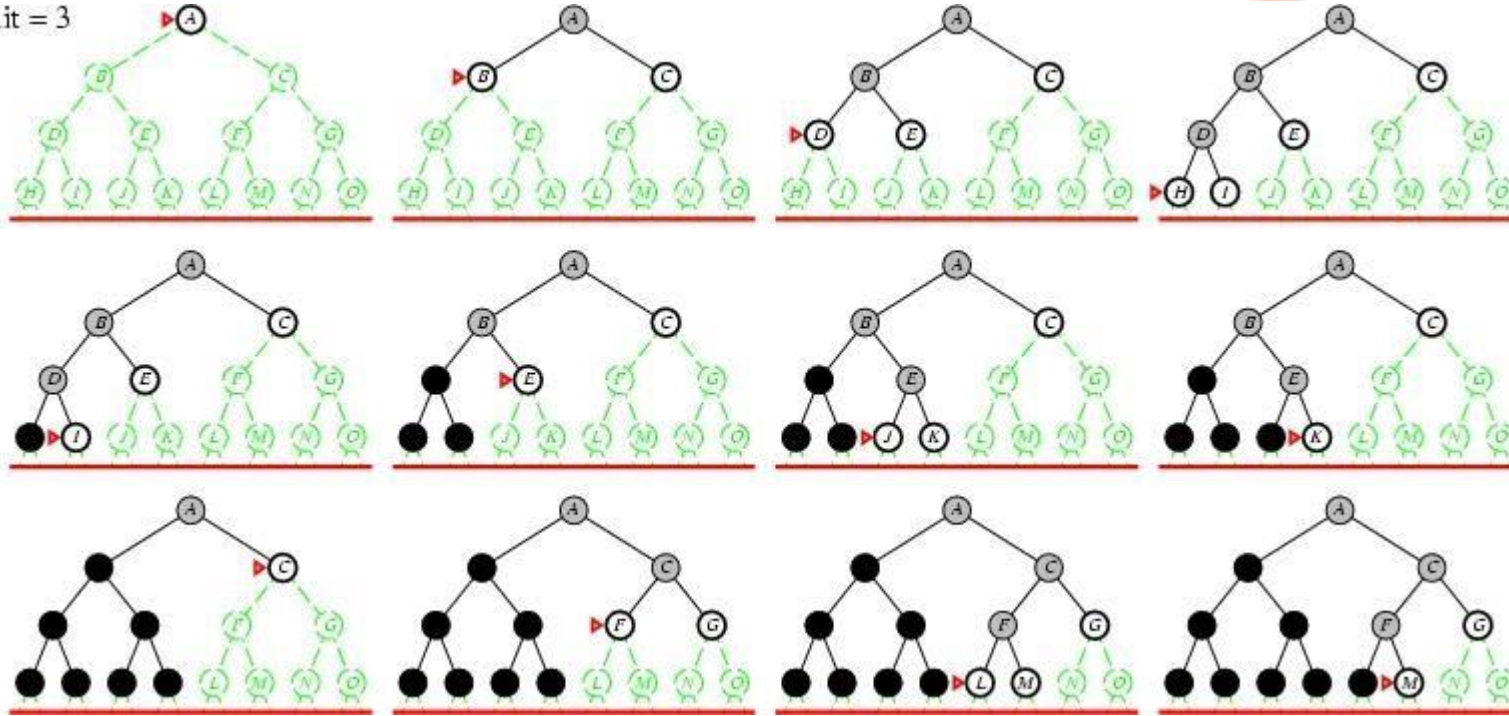
Iterative Deepening Search (IDS)

Limit = 2



Iterative Deepening Search (IDS)

Limit = 3



Iterative Deepening Search (IDS)

- IDS may seem wasteful as it is expanding the same nodes many times. In fact, when $b=10$ only about 11% more nodes are expanded than for a BFS
- Time Complexity = $O(b^d)$
- Space Complexity = $O(bd)$
- For large search spaces, where the depth of the solution is not known, IDS is the preferred uninformed search method

Iterative Deepening Search (IDS)

- Number of nodes generated in an iterative deepening search to depth d with branching factor b :
 - $N_{IDS} = (d+1)b^0 + d b^1 + (d-1)b^2 + (d-2)b^3 + (d-3)b^4 + b^d$
- In an iterative deepening search, the nodes on the bottom level (depth d) are generated once, those on the next-to-bottom level are generated twice, and so on, up to the children of the root, which are generated d times.

IDS

- For $b = 10$, $d = 5$,
 - – $N_{\text{BFS}} = 1 + 10 + 100 + 1,000 + 10,000 + 100,000 = 111,111$ –
 - – $N_{\text{IDS}} = 6 + 50 + 400 + 3,000 + 20,000 + 100,000 = 123,456$
- Overhead = $(123,456 - 111,111)/111,111 = 11\%$

IDS

- **Complete?** Yes
- **Time?** $O(b^d)$
- **Space?** $O(bd)$
- **Optimal?** Yes, if step cost = 1

Uniform-cost search

- When all step costs are equal, Breadth-First Search (BFS) is optimal because it always expands the shallowest unexpanded node.
- For problems with unequal step costs, Uniform-Cost Search (UCS) is used to ensure optimality.
- Uniform-Cost Search expands the node with the lowest path cost $g(n)$ rather than the shallowest depth.
- The frontier in UCS is maintained as a priority queue, ordered by path cost $g(n)$.

Uniform-cost search

- Algorithm

Insert the root into the queue

While the queue is not empty

 Dequeue the maximum priority element from the queue

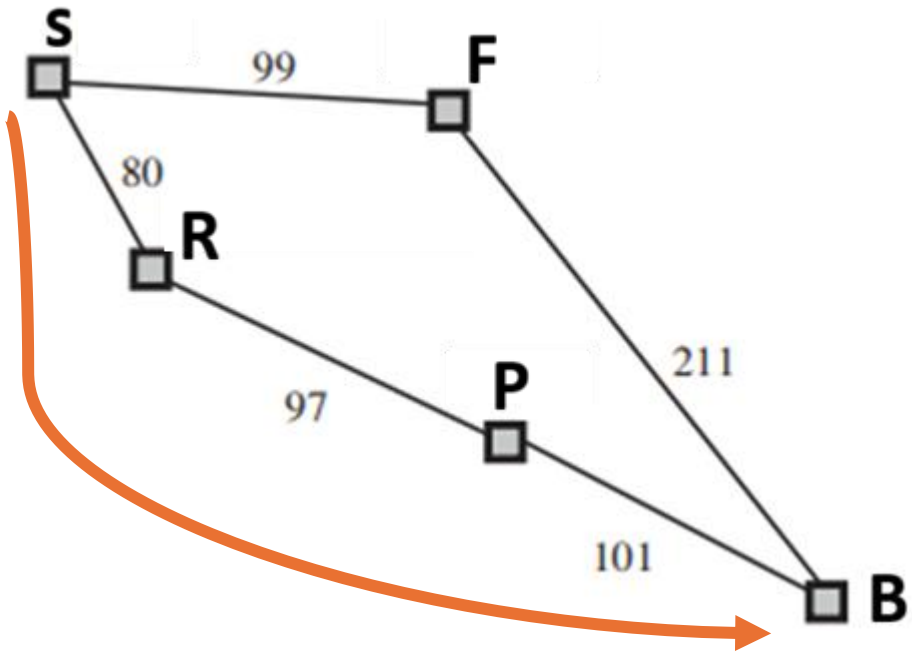
 (If priorities are same, alphabetically smaller path is chosen)

 If the path is ending in the goal state, print the path and exit

 Else

 Insert all the children of the dequeued element, with the cumulative costs as priority

Uniform-cost search



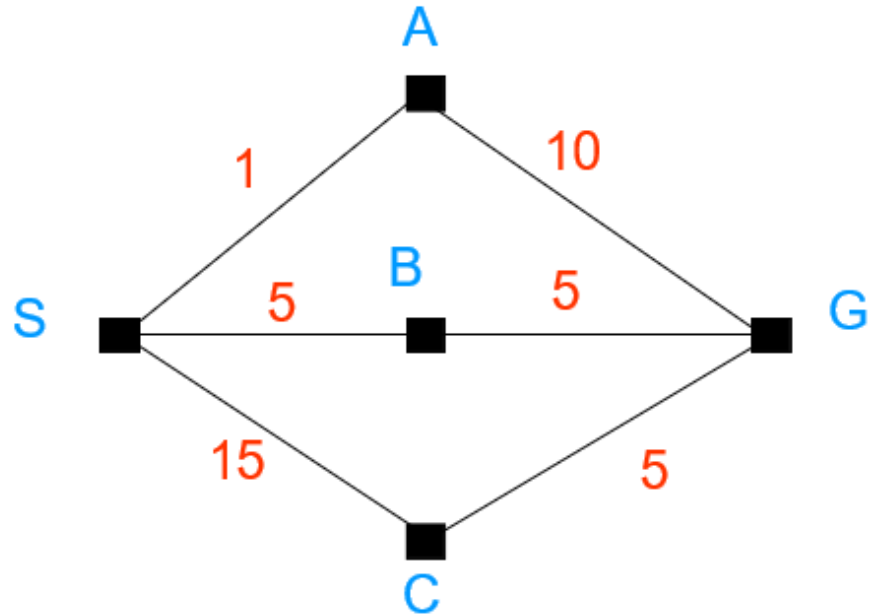
Path: $S \rightarrow R \rightarrow P \rightarrow B$

iter	Closed List	Opened List
1.	{}	{S(0)}
2.	{S}	{F(99), R(80)}
3.	{S, R}	{F(99), P(80+97=177)}
4.	{S, R, F}	{P(177), B1(99+211=310)}
5.	{S, R, F, P}	{B1(310), B2(177+101=278)}
6.	{S, R, F, P, B2}	

Uniform-cost search

- Uniform-cost search is guided by path costs rather than depths
- **Complete?** Yes, if step cost $\geq \epsilon$
- **Time?** # of nodes with $g(\text{path cost}) \leq \text{cost of optimal solution}$,
 $O(b^{1+\lceil C^*/\epsilon \rceil})$ where C^* is the cost of the optimal solution, ϵ
some small positive constant
- **Space?** # of nodes with $g \leq \text{cost of optimal solution}$ $O(b^{1+\lceil C^*/\epsilon \rceil})$
- **Optimal?** Yes – nodes expanded in increasing order of $g(n)$

UCS & BFS



- BFS will find the path SAG, with a cost of 11, but SBG is cheaper with a cost of 10
- Uniform Cost Search will find the cheaper solution (SBG).

Evaluation of tree-search strategies

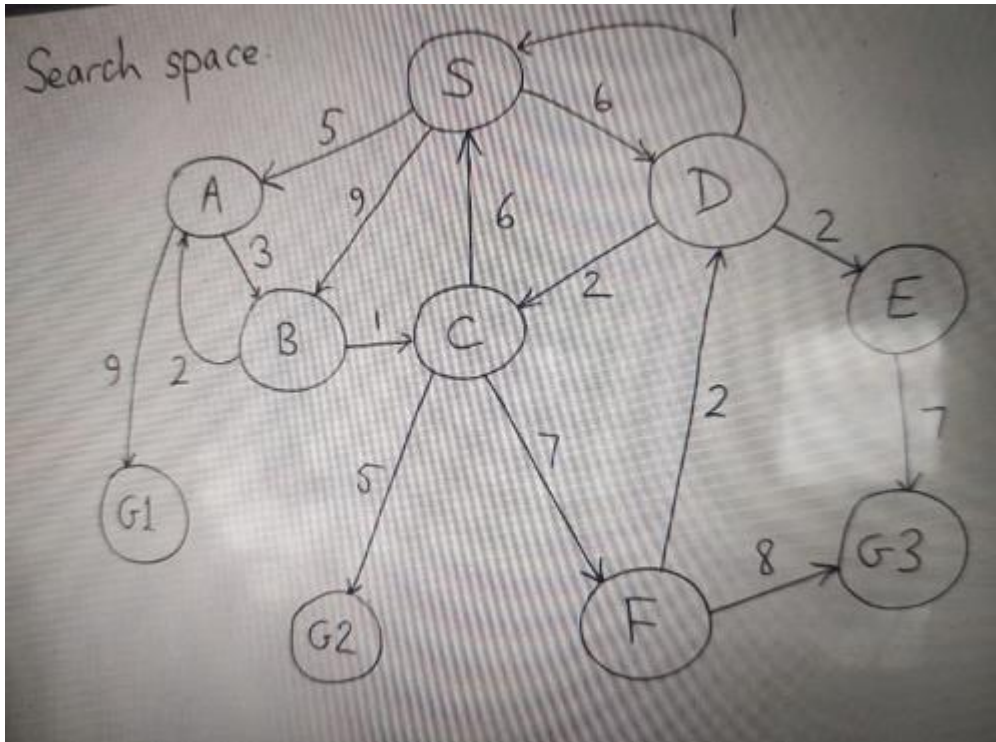
- b is the branching factor; d is the depth of the shallowest solution; m is the maximum depth of the search tree.

Criterion	Breadth-First	Uniform-Cost	Depth-First	Iterative Deepening
Complete?	Yes ^a	Yes ^{a,b}	No	Yes ^a
Time	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^d)$
Space	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(bd)$
Optimal?	Yes ^c	Yes	No	Yes ^c

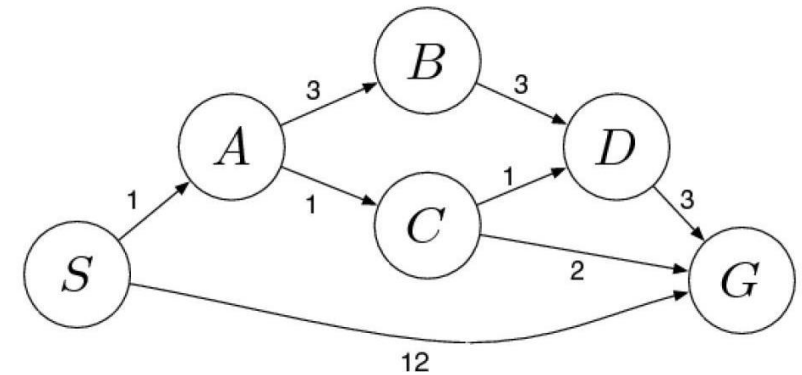
Superscript caveats are as follows: a complete if b is finite; b complete if step costs $\geq$ for positive ; c optimal if step costs are all identical.

Homework #2

(a)



(b)



Summary

- Problem formulation involves abstracting real-world details to define a manageable and searchable state space.
- There exists a variety of uninformed search strategies for exploring the state space.
- Iterative Deepening Search (IDS) uses linear space and requires only slightly more time than other uninformed search algorithms

References

- **CH 3- Solving Problems by Searching (Section 3.1-3.4.5)**